6 — Decision trees

Machine Learning (MPA-MLR)

Jiří Chmelík
Department of Biomedical Engineering · FEEC, Brno University of Technology

2026

Outline

1 · Bias, variance and noise

Where the test error comes from, and what we can do about it.

2 · Decision trees

Basic principle and terminology.

3 · CART

Classification And Regression Trees, Breiman et al. (1984): splitting criterion, stop-splitting rule, leaf assignment rule.

4 · Bootstrap aggregated (bagged) trees

Many trees on resampled data, one average.

5 · Random forest

Bagging with a random subset of features per split.

6 · Adaptive boosting (AdaBoost)

Weak classifiers trained one after another, weighted vote.

7 · Other tree algorithms and pruning

ID3, C4.5 / C5.0, CHAID, MARS, pruning.

1 · Bias, variance and noise

> Before we build a single tree, we split the test error into three parts — and two of them are ours to control.



Illustration generated by AI (FLUX.2)

Bias–variance tradeoff and expected test error

Training data — n samples drawn i.i.d. from $P(X, Y)$:

$$T = \{(\mathbf{x}_1, y_1), \ldots, (\mathbf{x}_n, y_n)\}, \qquad \mathbf{x}_i \in \mathbb{R}^d, \quad y_i \in \mathbb{R}$$

Expected label (given the feature vector $\mathbf{x}$):

$$\bar{y}(\mathbf{x}) = E_{y|\mathbf{x}}[Y] = \int_y y\, P(y \mid \mathbf{x})\, dy$$

Expected test error (given a trained model h_T):

$$E_{(\mathbf{x},y)\sim P}\left[(h_T(\mathbf{x}) - y)^2\right] = \iint_{\mathbf{x}y} (h_T(\mathbf{x}) - y)^2\, P(\mathbf{x}, y)\, dy\, d\mathbf{x}$$

Training process — a learning algorithm $\mathcal{A}$ turns a dataset into a model:

$$h_T = \mathcal{A}(T)$$

Expected model (given the algorithm $\mathcal{A}$, averaged over all datasets):

$$\bar{h} = E_{T\sim P^n}\left[h_T\right] = \int_T h_T\, P(T)\, dT$$

Expected test error: adding and subtracting the average model

Now average over datasets as well:

$$E_{\substack{(\mathbf{x},y)\sim P \\ T\sim P^n}}\left[(h_T(\mathbf{x})-y)^2\right] = \iiint_{T\mathbf{x}y} (h_T(\mathbf{x})-y)^2\, P(\mathbf{x},y)P(T)\,dy\,d\mathbf{x}\,dT$$

Add and subtract $\bar{h}(\mathbf{x})$ and expand with $(a+b)^2 = a^2+2ab+b^2$:

$$E_{\mathbf{x},y,T}\left[(h_T(\mathbf{x})-y)^2\right] = E_{\mathbf{x},y,T}\left[\left[\underbrace{(h_T(\mathbf{x})-\bar{h}(\mathbf{x}))}_{a} + \underbrace{(\bar{h}(\mathbf{x})-y)}_{b}\right]^2\right]$$

$$= E_{\mathbf{x},T}\left[(h_T(\mathbf{x})-\bar{h}(\mathbf{x}))^2\right] + \underbrace{2E_{\mathbf{x},y,T}\left[(h_T(\mathbf{x})-\bar{h}(\mathbf{x}))(\bar{h}(\mathbf{x})-y)\right]}_{=0} + E_{\mathbf{x},y}\left[(\bar{h}(\mathbf{x})-y)^2\right]$$

The cross term vanishes because $E_T[h_T(\mathbf{x})] = \bar{h}(\mathbf{x})$, so

$$\boxed{E_{\mathbf{x},y,T}\left[(h_T(\mathbf{x})-y)^2\right] = E_{\mathbf{x},T}\left[(h_T(\mathbf{x})-\bar{h}(\mathbf{x}))^2\right] + E_{\mathbf{x},y}\left[(\bar{h}(\mathbf{x})-y)^2\right]}$$

Variance, bias and noise

The same trick once more, now with the expected label $\bar{y}(\mathbf{x})$:

$$E_{\mathbf{x},y}\left[(\bar{h}(\mathbf{x}) - y)^2\right] = E_{\mathbf{x}}\left[(\bar{h}(\mathbf{x}) - \bar{y}(\mathbf{x}))^2\right] + \underbrace{2E_{\mathbf{x},y}\left[(\bar{h}(\mathbf{x}) - \bar{y}(\mathbf{x}))(\bar{y}(\mathbf{x}) - y)\right]}_{=\,0} + E_{\mathbf{x},y}\left[(\bar{y}(\mathbf{x}) - y)^2\right]$$

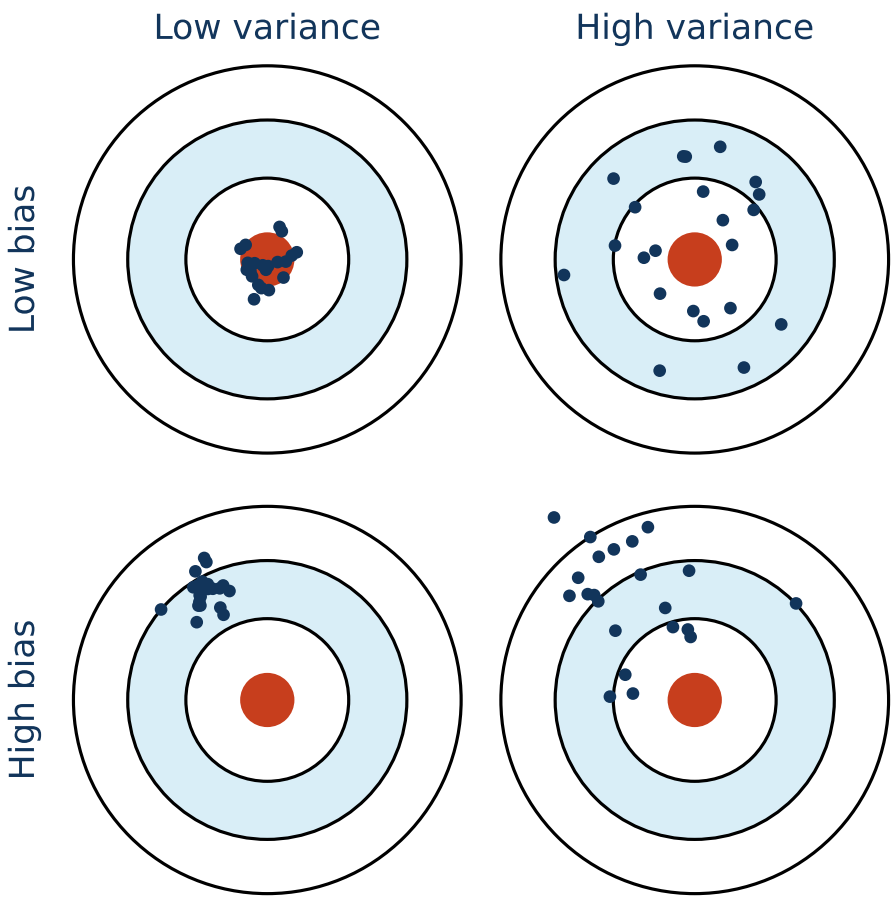
Putting both steps together:

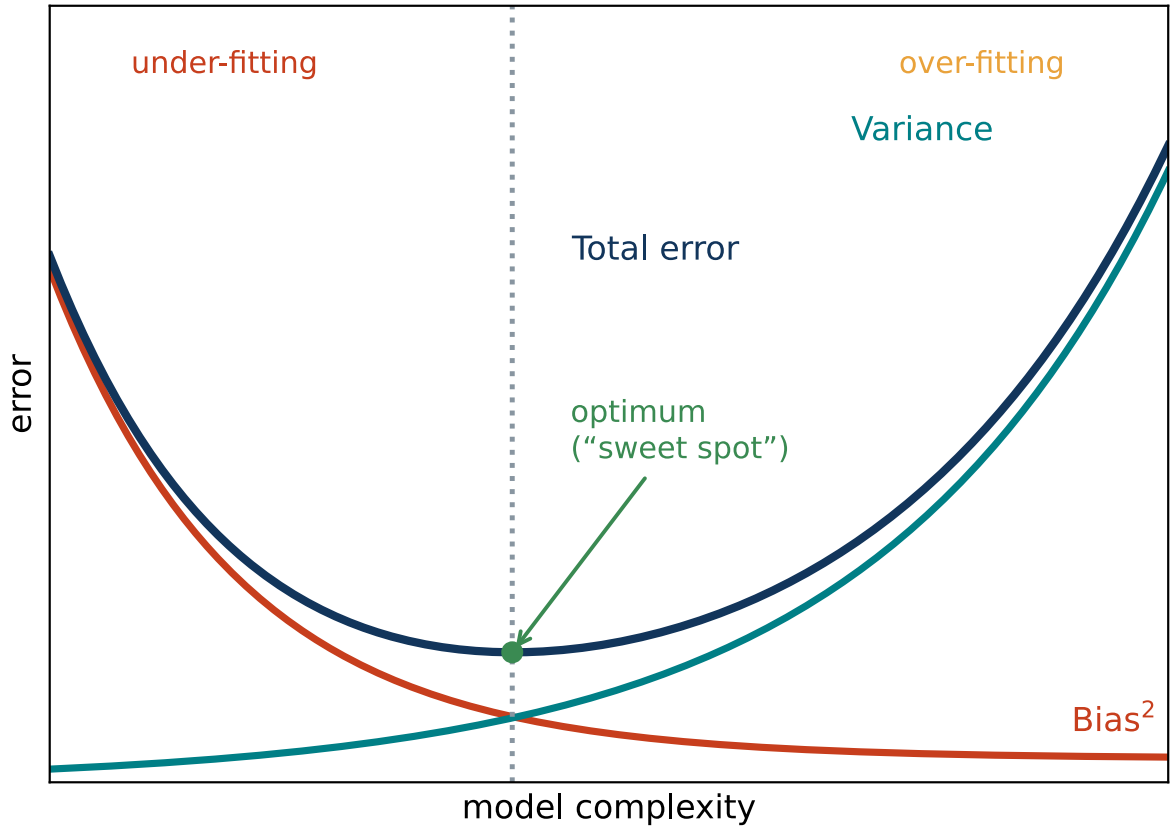
$$\underbrace{E_{\mathbf{x},y,T}\left[(h_T(\mathbf{x}) - y)^2\right]}_{\textbf{expected test error}} = \underbrace{E_{\mathbf{x},T}\left[(h_T(\mathbf{x}) - \bar{h}(\mathbf{x}))^2\right]}_{\textbf{variance}} + \underbrace{E_{\mathbf{x}}\left[(\bar{h}(\mathbf{x}) - \bar{y}(\mathbf{x}))^2\right]}_{\textbf{bias}^2} + \underbrace{E_{\mathbf{x},y}\left[(\bar{y}(\mathbf{x}) - y)^2\right]}_{\textbf{noise}}$$

- **Variance** — how much the model changes when the training set changes.
- **Bias** — how far the *average* model is from the best possible prediction.
- **Noise** — irreducible; no algorithm and no amount of data can remove it.

Details: cs4780 lecture note 12

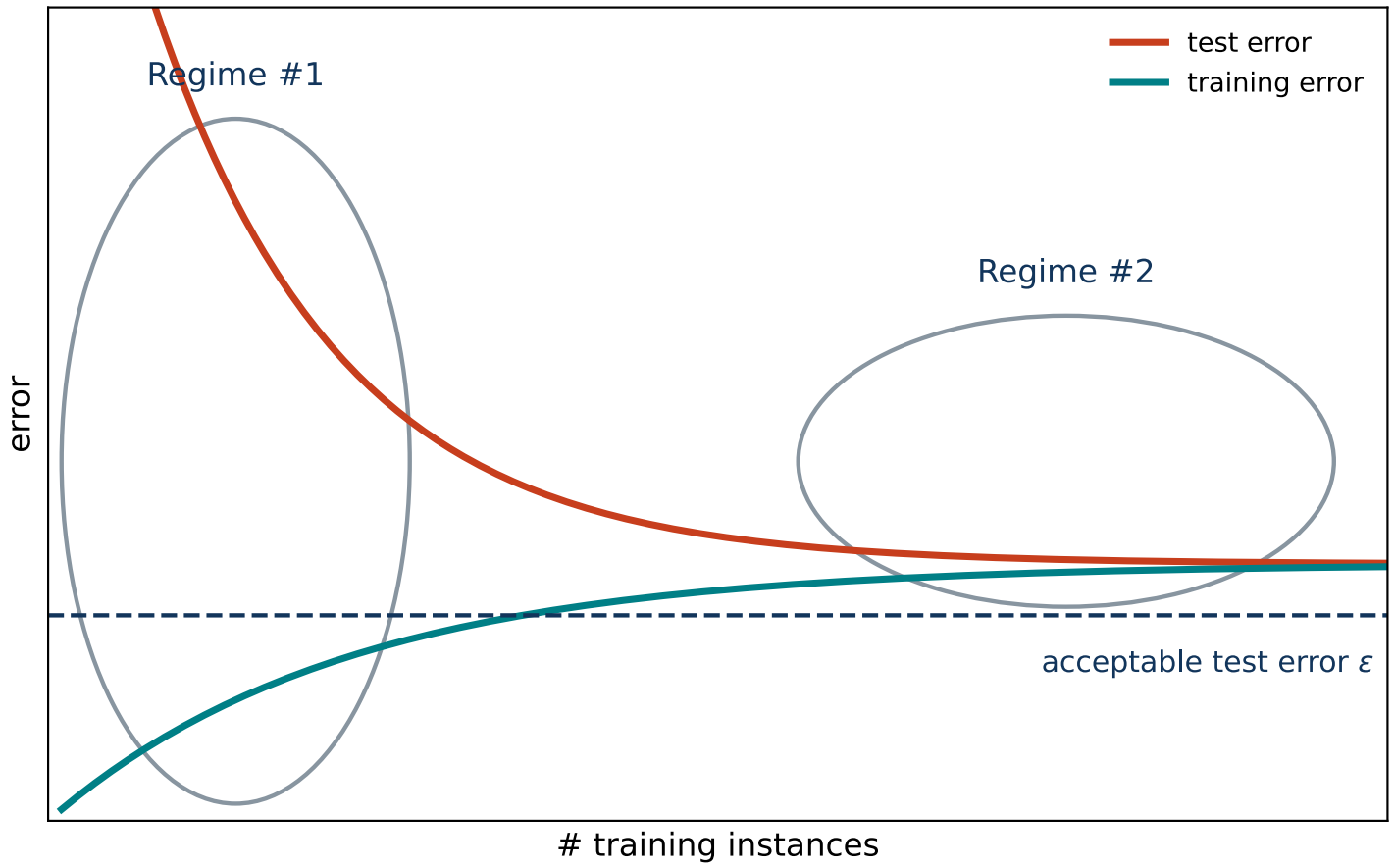
Bias and variance visually





Source: scott.fortmann-roe.com/docs/BiasVariance.html

Learning curves: two regimes



Fix an **acceptable test error** ϵ (given by the application, not by the data) and watch both curves as the training set grows.

The two regimes need opposite treatments, so first find out which one you are in.

Source: cs4780 lecture note 12

Regime #1 — high variance

How to recognize?

- Training error is much lower than test error.
- Training error is lower than ϵ.
- Test error is higher than ϵ.

Suggestions

- Add more training data.
- Reduce model complexity.
- Bootstrap aggregation (“bagging”).



Illustration generated by AI (FLUX.2)

The model fits the training set well but a different training set would give a very different model — the gap between the curves *is* the variance term.

Regime #2 — high bias

How to recognize?

- Training error is higher than ϵ (and the test error is close to it).

Suggestions

- Use a more complex model.
- Add features.
- Boosting.

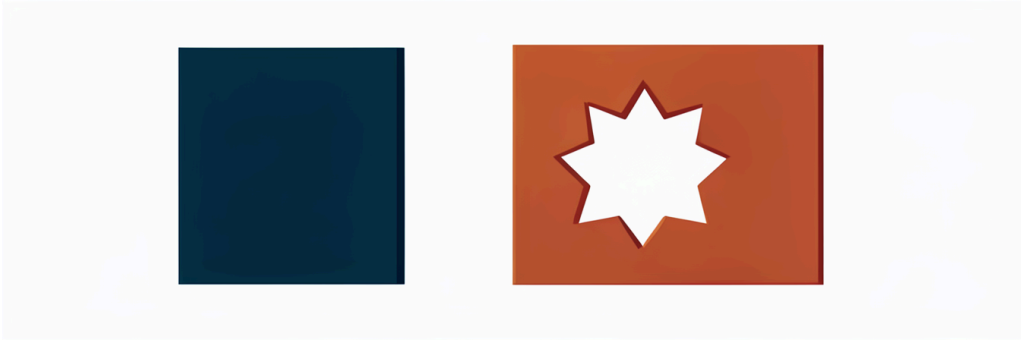


Illustration generated by AI (FLUX.2)

More data does **not** help here: both curves have already converged, and they converged to the wrong place.

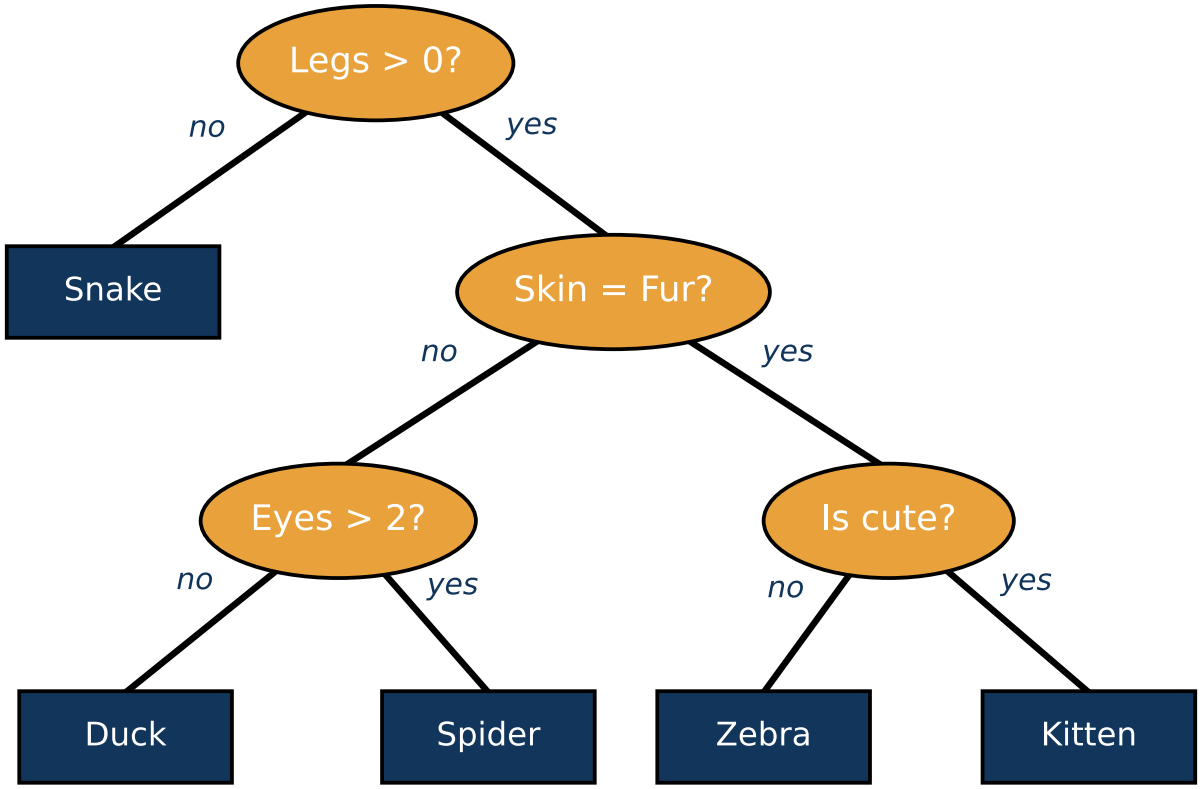
2 · Decision trees

> A sequence of yes/no questions is a model — and one anybody can read.



Illustration generated by AI (FLUX.2)

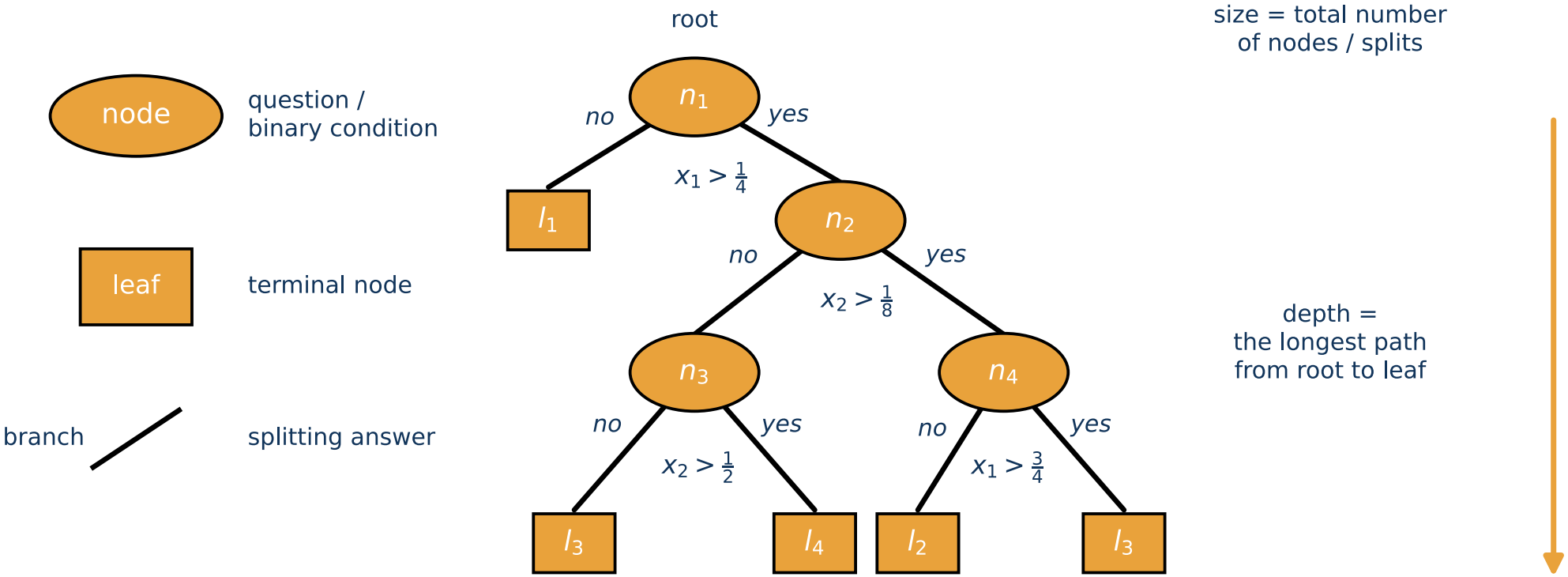
Decision trees — basic principle



Animal	Skin	Legs	Eyes	Cute
Kitten	Fur	4	2	True
Snake	Scales	0	2	False
Spider	Chitin	8	8	False
Zebra	Fur	4	2	False
Duck	Feather	2	2	False

Each **node** asks one binary question about one feature; each **leaf** carries the answer. A sample is classified by walking from the root to a leaf.

Decision trees — terminology



note: a tree with one node only — “stump”

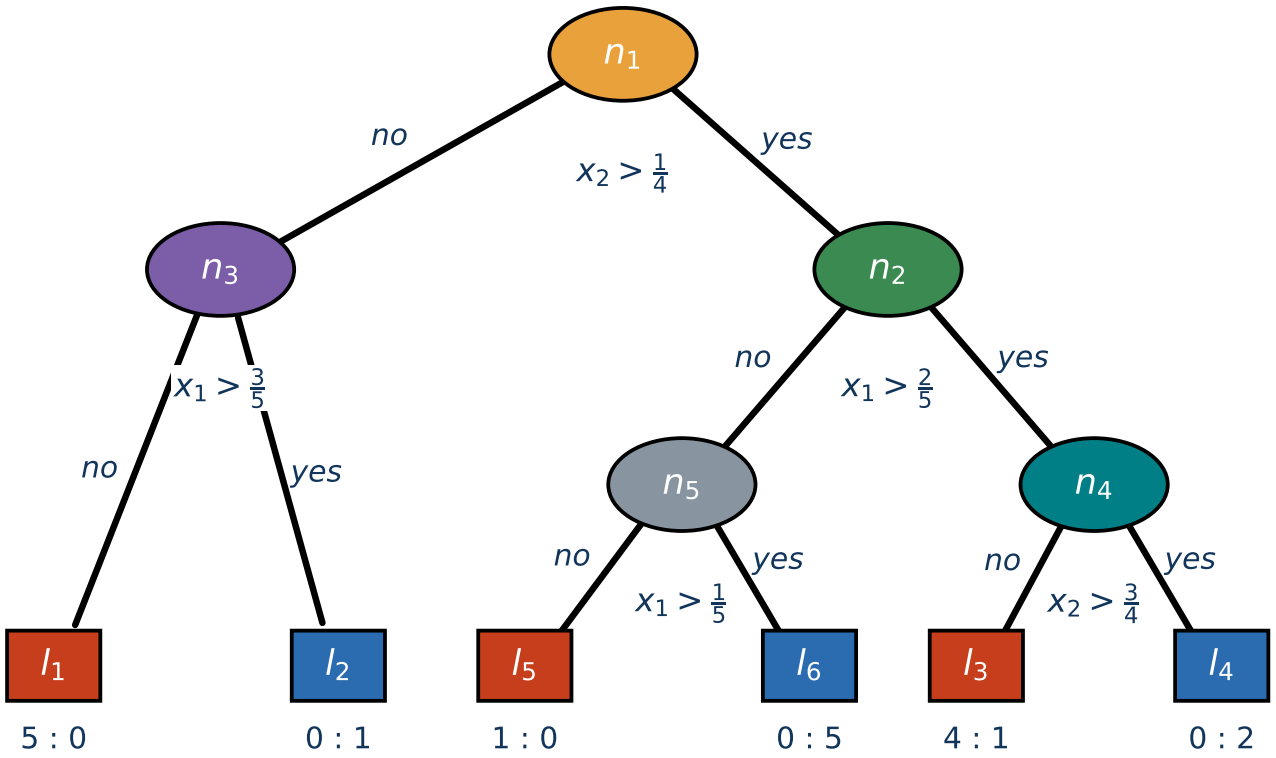
3 · CART

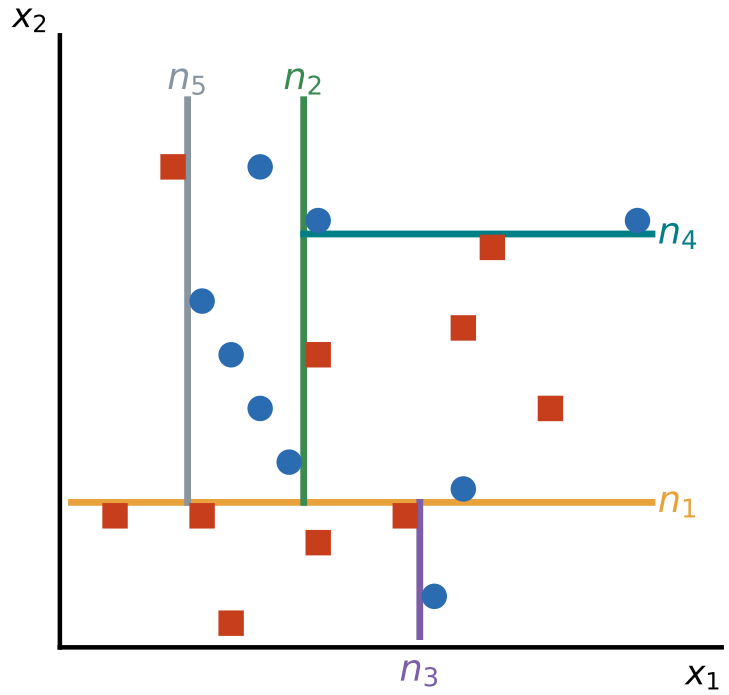
> Three decisions make a tree algorithm: where to split, when to stop, and what to put in a leaf.



Illustration generated by AI (FLUX.2)

CART — classification





$T = \{(\mathbf{x}_1, y_1), \ldots, (\mathbf{x}_n, y_n)\}, \quad \mathbf{x}_i = (x_{i1}, \ldots, x_{id}), \quad x_{ij} \in \mathbb{R}, \quad y_i \in Y$ (Y — set of classes)

Each node splits **one** feature by a threshold, so the tree carves the feature space into axis-aligned rectangles. Each leaf is assigned one class by **majority vote**; the numbers below the leaves are the class counts. This is exactly the tree that CART grows on these 19 points — the next slides show how.

Growing a classification tree: the splitting criterion

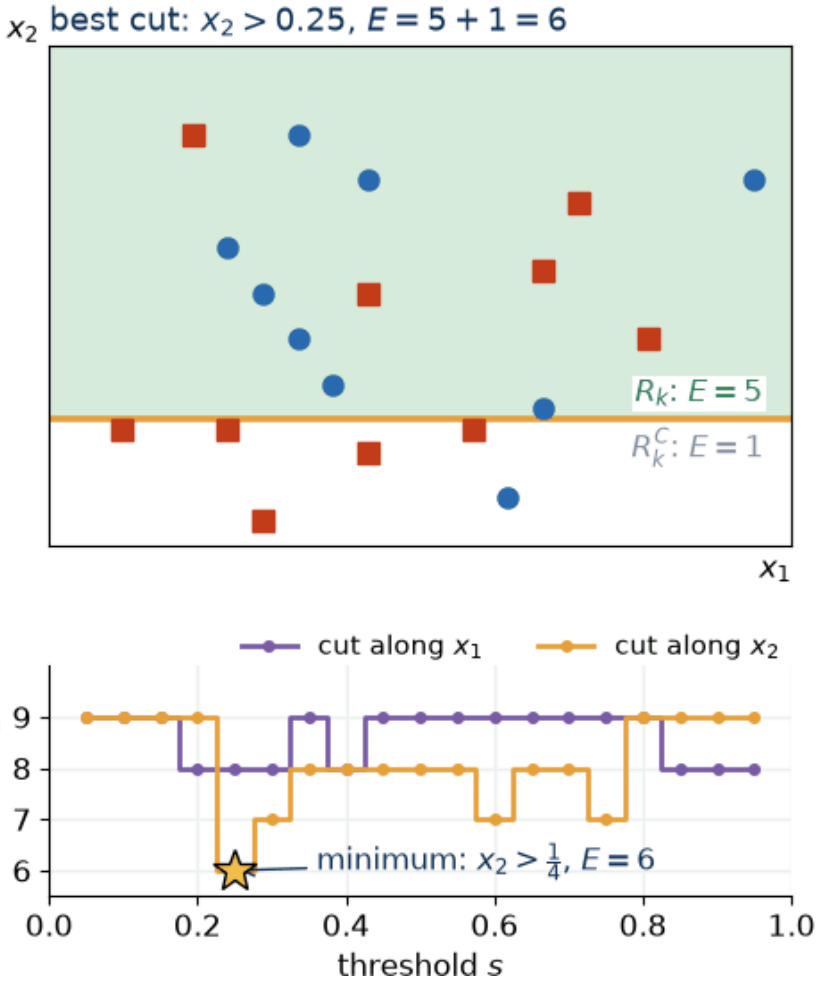
Task: minimize the classification error E_k (number of misclassified points) in the k-th node, which is split into two complementary regions R_k and R_k^C by a threshold s on the j-th feature:

$$E_k = \min_{\substack{j \in 1,2,\ldots,d \\ s \in \mathbb{R}}} \left[E_{R_k(j,s)} + E_{R_k^C(j,s)} \right]$$

$$R_k(j,s) = \{\mathbf{x}_i : x_{ij} > s\}, \qquad R_k^C(j,s) = \{\mathbf{x}_i : x_{ij} \leq s\}$$

$$E_{R_k} = \sum_{i:\, \mathbf{x}_i \in R_k} I(y_i \neq \hat{y}_{R_k})$$

$\hat{y}_{R_k}$ — the majority class of the region, $I(\cdot)$ — indicator function

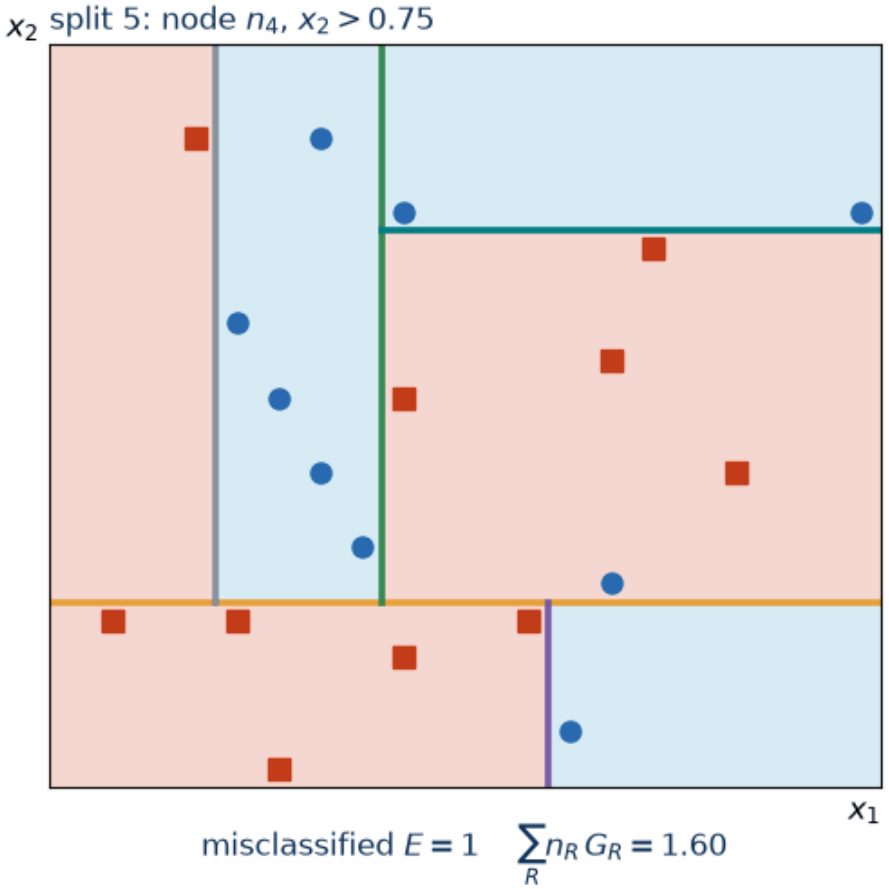


Growing a classification tree: greedy search

“Greedy” approach during tree growing

1. Compute the error E_k for each region R_k by selecting a feature j and a threshold s (by brute force or histogram-based methods).
2. Take the best region (with the lowest error) and split the data into two sub-regions in accordance with this feature and threshold.
3. Repeat for each new sub-region until a stop-splitting criterion is reached.

On the 19 points of the example: five splits in the order n_1, n_3, n_2, n_5, n_4; the number of misclassified points goes $9 \to 6 \to 5 \to 4 \to 3 \to 1$ and the Gini sum $\sum_R n_R G_R$ goes $9.47 \to \cdots \to 1.60$. The last leaf l_3 (4 : 1) stays impure — the depth limit `max_depth = 3` stops the growth.

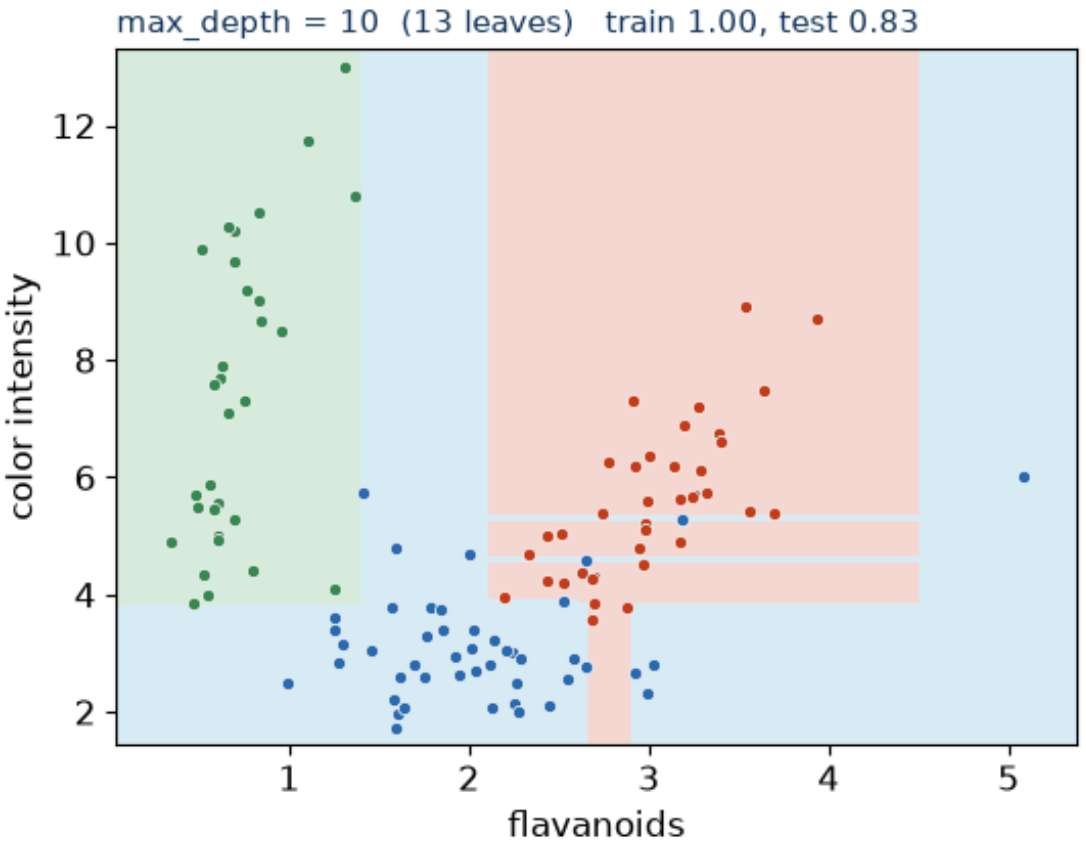


Growing a classification tree: stop-splitting rules

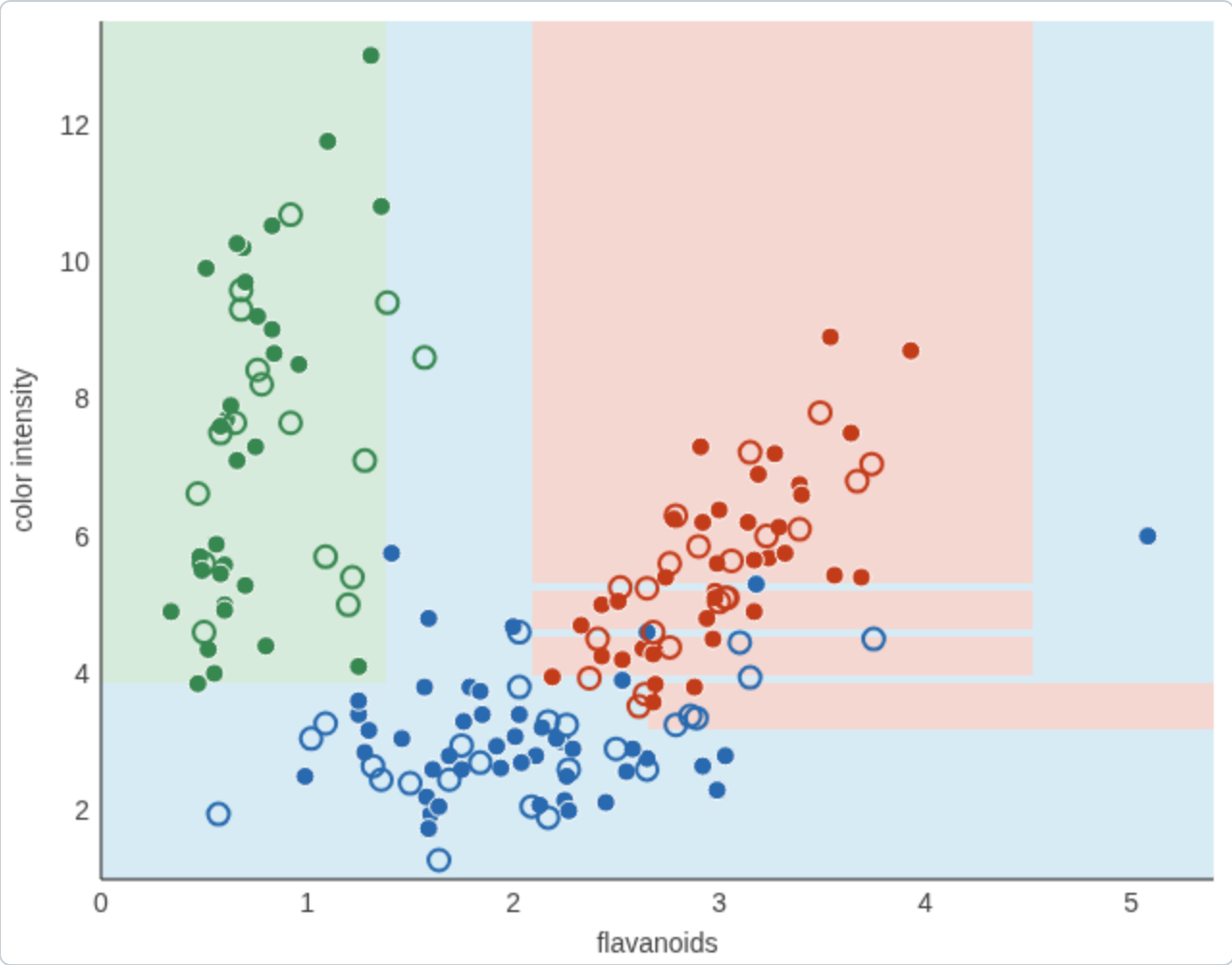
Stop-splitting rules

- when there is one point in the region,
- split only if there are more than n_{min} points in the region,
- when the region contains points of one class only,
- maximum depth of the tree reached,
- maximum number of splits (tree size) reached.

Without any of them the tree grows until every leaf is pure — the animation shows `max_depth` from 1 to 10 on two features of the *wine* dataset: the training accuracy climbs to 1.00, the test accuracy peaks at depth 2 (0.87) and ends at 0.83.



Playground: depth and leaf size



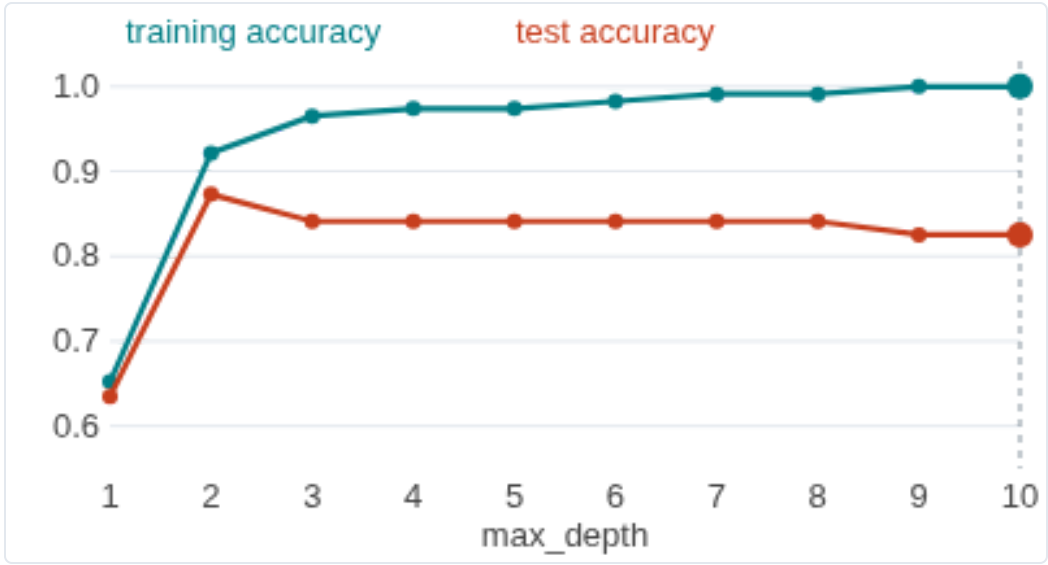
wine, two features, 115 training points (filled) and 63 test points (rings). The CART tree is regrown on every change.

`max_depth` = **10**

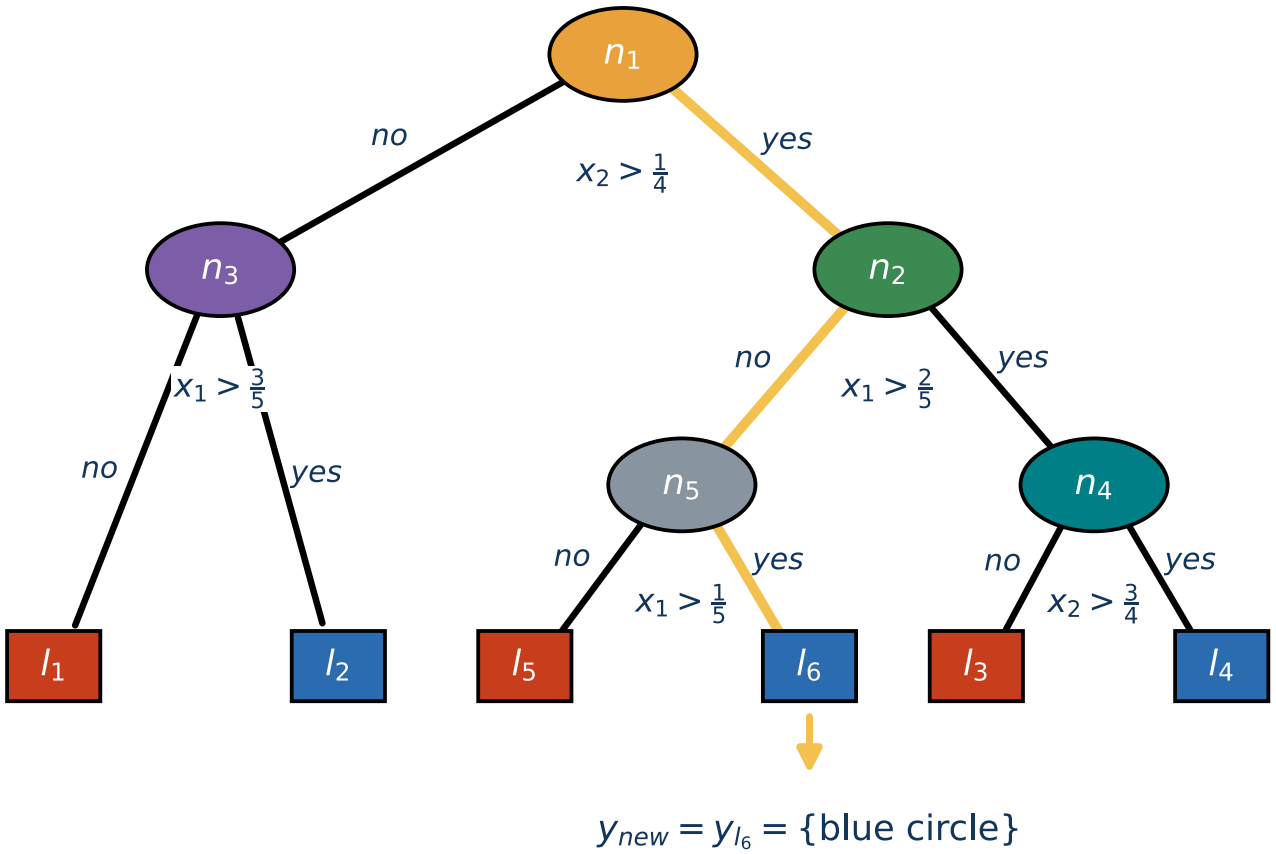
`min_samples_leaf` = **1**

☑ test points

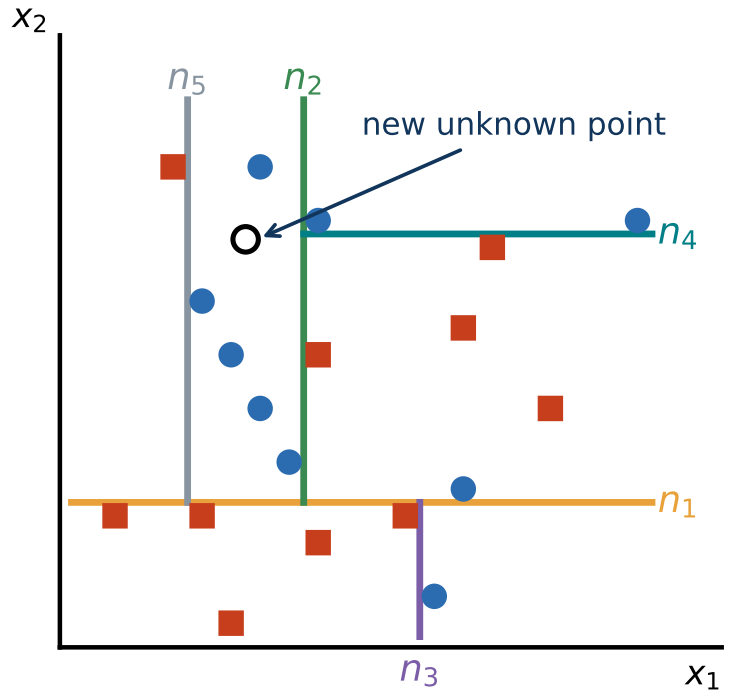
13 leaves · training accuracy **1.00** · test accuracy **0.83**



CART — classifying a new sample

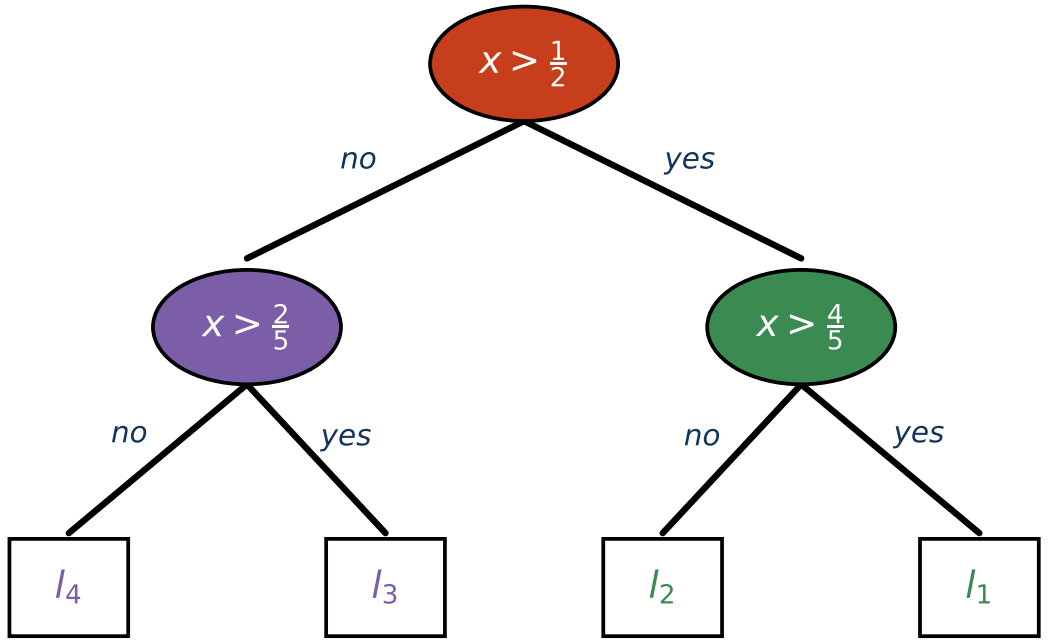


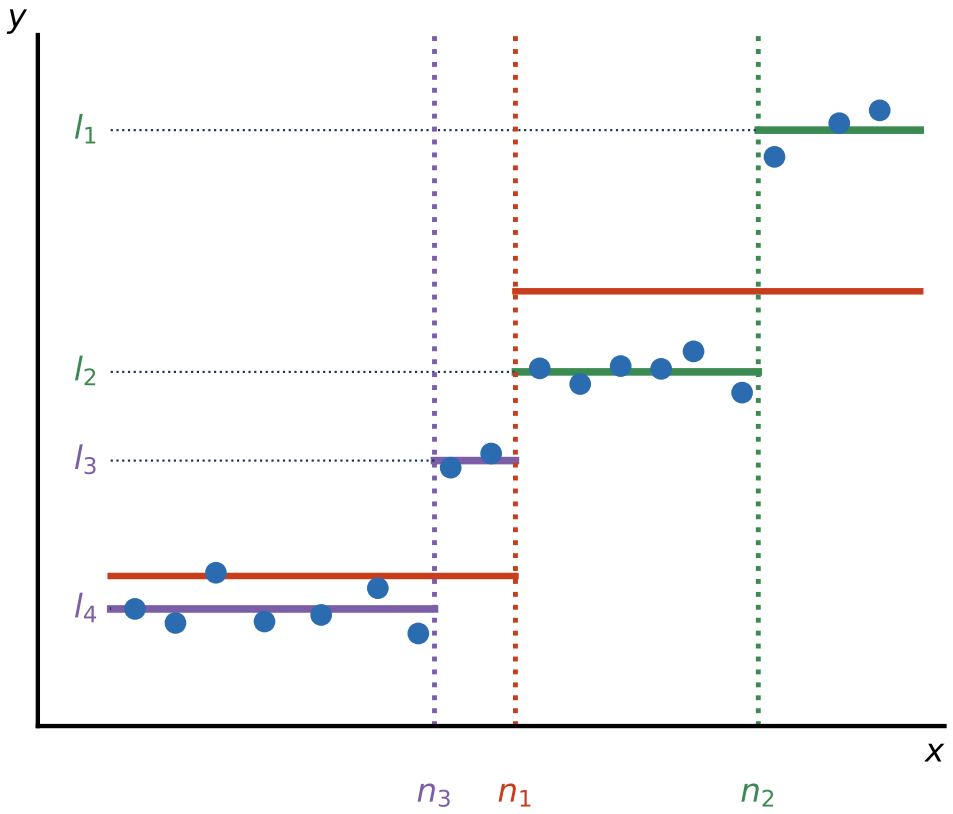
$$y_{new} = y_{l_6} = \{\text{blue circle}\}$$



Prediction is a walk from the root to a leaf: answer one question per node. The cost is the **depth** of the tree, not the number of training samples.

CART — regression





Same procedure, different splitting criterion — **squared error**:

$$E_{R_k} = \sum_{i:\, \mathbf{x}_i \in R_k} (y_i - \hat{y}_{R_k})^2$$

The value $\hat{y}_{R_k}$ of each leaf is the **mean** of the points that fall into it, so the model is a piecewise-constant (staircase) function.

CART — impurity measures

Classification — $p_{R_k}(y)$ is the fraction of class y in the region

Misclassification error (count; the rate $E_{R_k}/n_{R_k} = 1 - \max_y p_{R_k}(y)$ is plotted)

$$E_{R_k} = \sum_{i:\, \mathbf{x}_i \in R_k} I(y_i \neq \hat{y}_{R_k})$$

Entropy

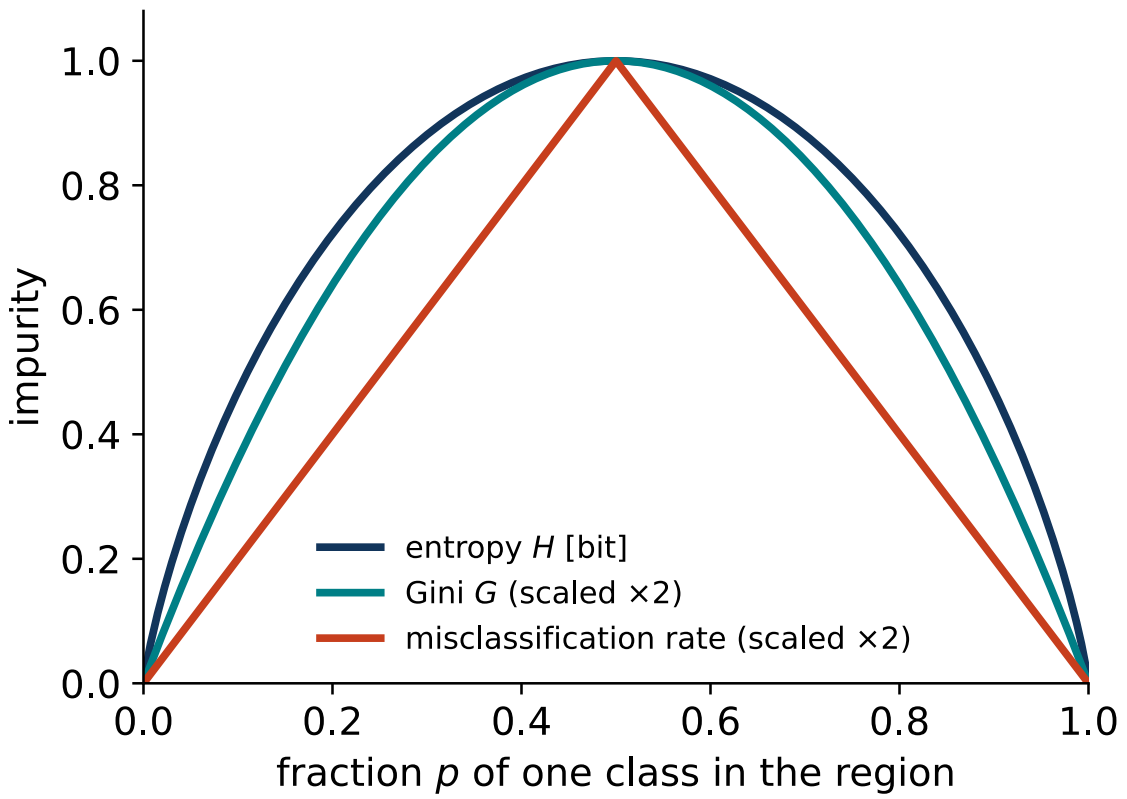
$$H_{R_k} = -\sum_{y \in Y} p_{R_k}(y) \cdot \log\left(p_{R_k}(y)\right)$$

“Gini impurity”

$$G_{R_k} = \sum_{y \in Y} p_{R_k}(y) \cdot \left(1 - p_{R_k}(y)\right)$$

Regression — squared error

$$E_{R_k} = \sum_{i:\, \mathbf{x}_i \in R_k} (y_i - \hat{y}_{R_k})^2$$



By hand, two classes, $\log_2$: leaf l_3 $(4:1)$, $p = 0.8$: $E = 1$ error (rate 0.2), $H = 0.72$ bit, $G = 0.32$. Root $(10:9)$: $H = 1.00$ bit, $G = 0.50$. A pure leaf: all three are zero.

CART — conclusion

Pros

- simple and intuitive,
- can handle numerical and categorical data,
- minimal requirements for data preprocessing (independent of scaling — no need for normalization),
- can handle missing values (surrogate splits),
- can handle correlated data.

Cons

- instability — a small change in the data rewrites the whole tree (remedy: bagging, random forest),
- overfitting of deep trees (remedy: stopping rules, pruning, bagging, random forest).



Illustration generated by AI (FLUX.2)

code example — depth of the tree and the decision boundary

4 · Bootstrap aggregated (bagged) trees

> One tree is unstable. The average of many trees is not.

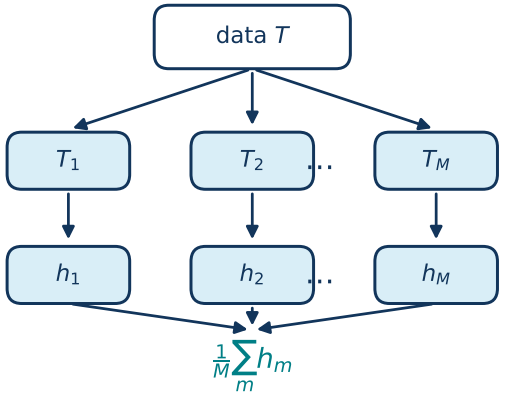


Illustration generated by AI (FLUX.2)

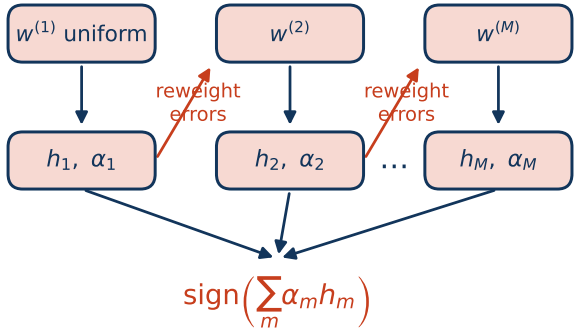
Ensemble methods

- Combine many **base models** (here trees) into one stronger model.
- **B**ootstrap **Agg**regation (“Bagging”) — simple average.
- **Ada**ptive **Boost**ing (“AdaBoost”) — weighted average.
- And many modifications: e.g. LPBoost, RUSBoost, RobustBoost, XGBoost, etc.

Bagging — independent models, simple average



Boosting — models in sequence, weighted sum



Bagging attacks the variance, boosting attacks the bias.

Bootstrap aggregation — the variance of a single tree

Standard regression tree trained on the whole dataset T:

$$T = \{(x_1, y_1), \ldots, (x_n, y_n)\}, \quad x_i \in \mathbb{R},\ y_i \in \mathbb{R},\ \text{i.i.d.}$$

Variance of its prediction at a point x over training sets:

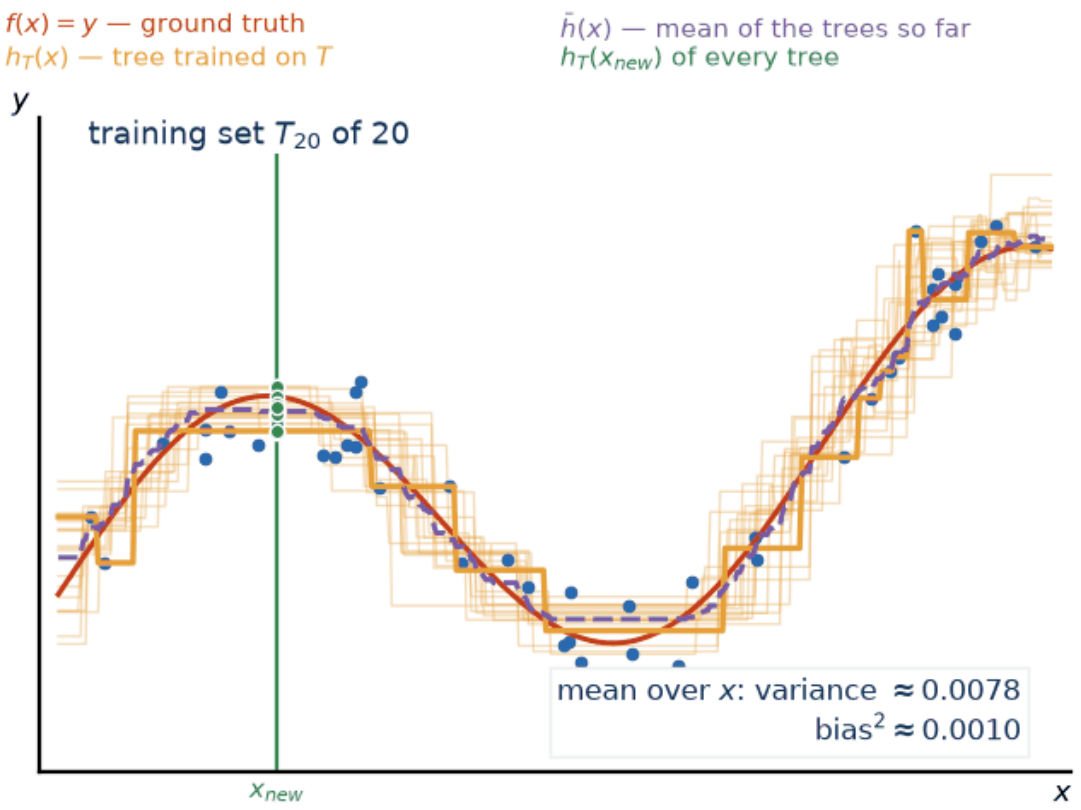
$$\boxed{\mathrm{Var}_T\left[h_T(x)\right] = E_T\left[\left(h_T(x) - \bar{h}(x)\right)^2\right]}$$

High variance problem:

$$E_{x,T}\left[\left(h_T(x) - \bar{h}(x)\right)^2\right] \to \text{high}$$



Illustration generated by AI (FLUX.2)



Bootstrap aggregation — bagging

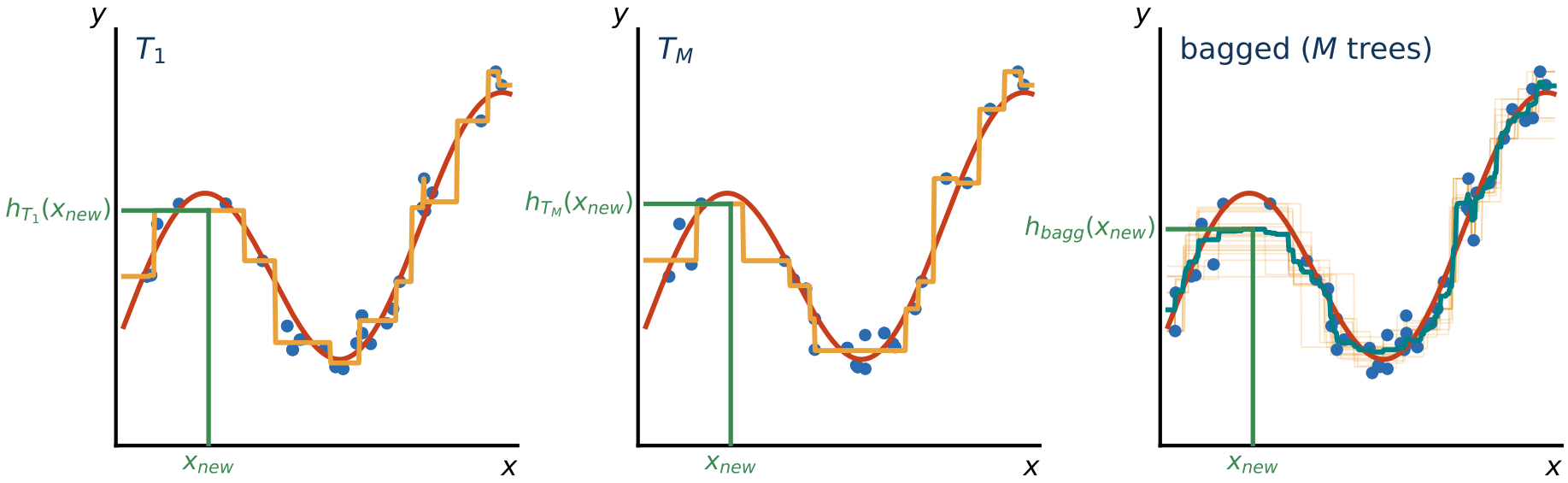
- Statistical technique to reduce the **variance** of a classification or regression procedure.
- Take $M \times n'$ random samples **with replacement** from T with uniform distribution:

$$T_1 = \left\{ \left(x_1^{(1)}, y_1^{(1)}\right), \ldots, \left(x_{n'}^{(1)}, y_{n'}^{(1)}\right) \right\}, \quad \ldots, \quad T_M$$

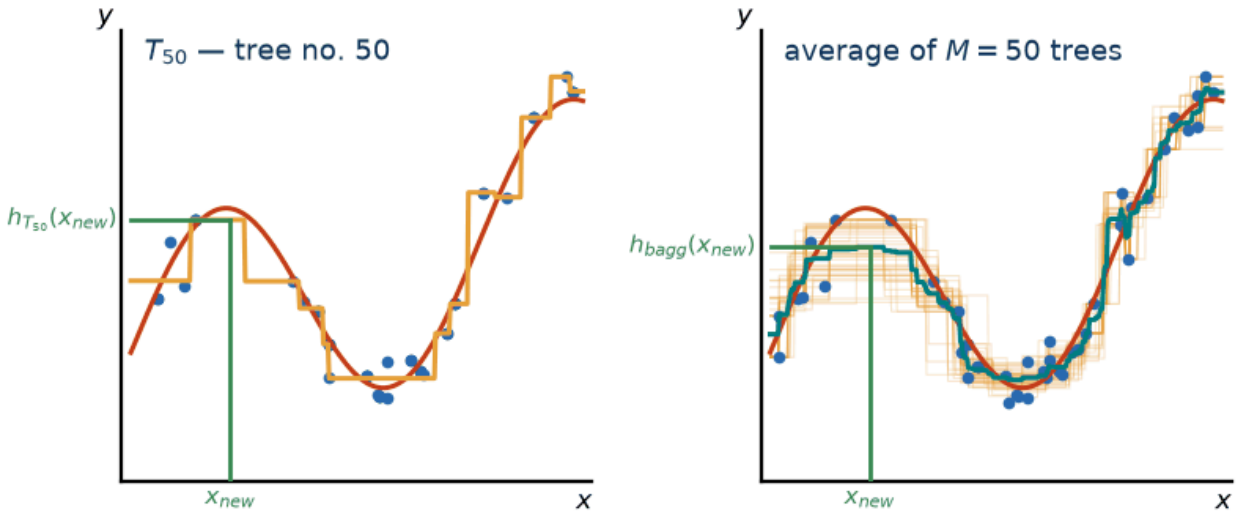
$$h_{bagg}(x) = \frac{1}{M} \sum_{m=1}^{M} h_{T_m}(x)$$

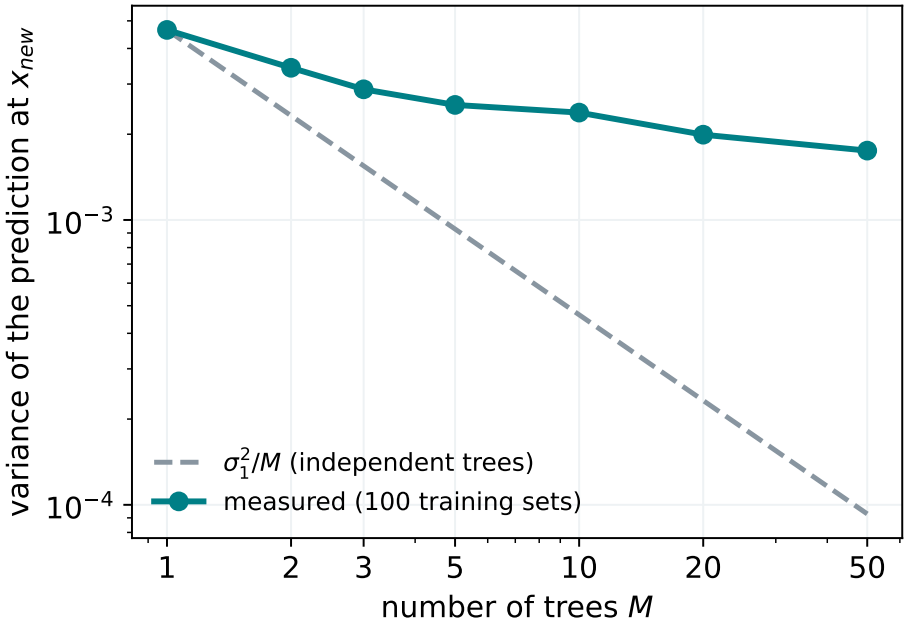
$$\boxed{\mathrm{Var}\left[h_{bagg}(x)\right] = \frac{1}{M}\sigma_1^2} \quad \text{(independent trees)}$$

σ_1^2 — variance of one tree in the average. Usually $n' = n$; regression — average, classification — majority vote.



Bagging step by step: does the variance drop as 1/M?



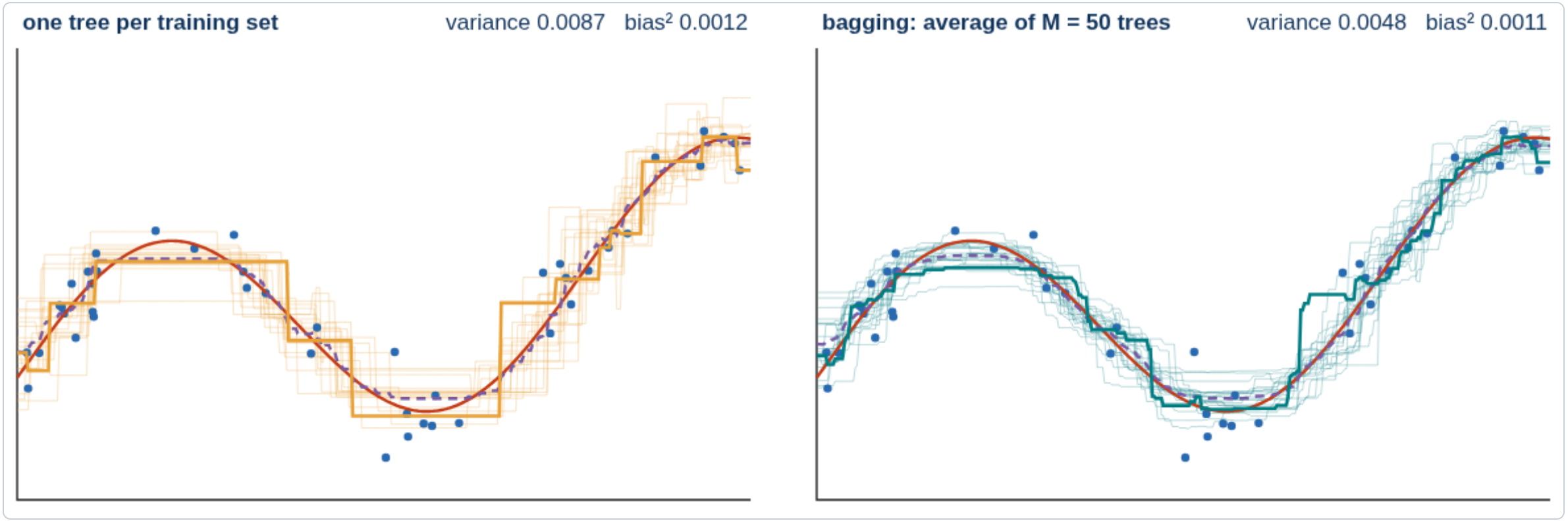


Left: trees added one at a time, the average settles. Right: the variance of the prediction at x_{new} over 100 independent training sets of $n = 45$ points. From one *bootstrap* tree to $M = 50$ it drops from 0.00465 to 0.00175 — a factor of **2.7, not 50**: the trees share the same data, so they are correlated (correlation ρ):

$$\mathrm{Var}\left[h_{bagg}(x)\right] = \rho\,\sigma_1^2 + \frac{1-\rho}{M}\,\sigma_1^2$$

code example — variance of one tree vs. bagging

Playground: one tree vs. bagging



tree `max_depth` = **4**

number of bagged trees **M** = **50**

+1 training set | +10 training sets | reset

20 training sets of n = 45 · bagging divides the variance by **1.8**

— truth - - mean over the training sets (≈ $\bar{h}$) ● the last training set

5 · Random forest

> Bagging plus one extra dose of randomness: each split sees only a few features.

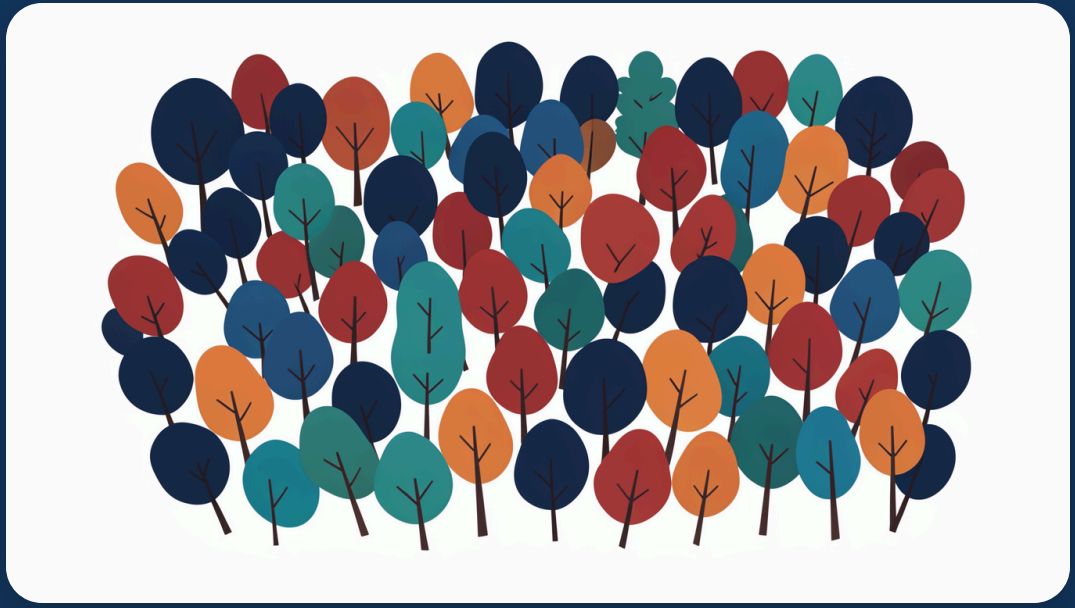
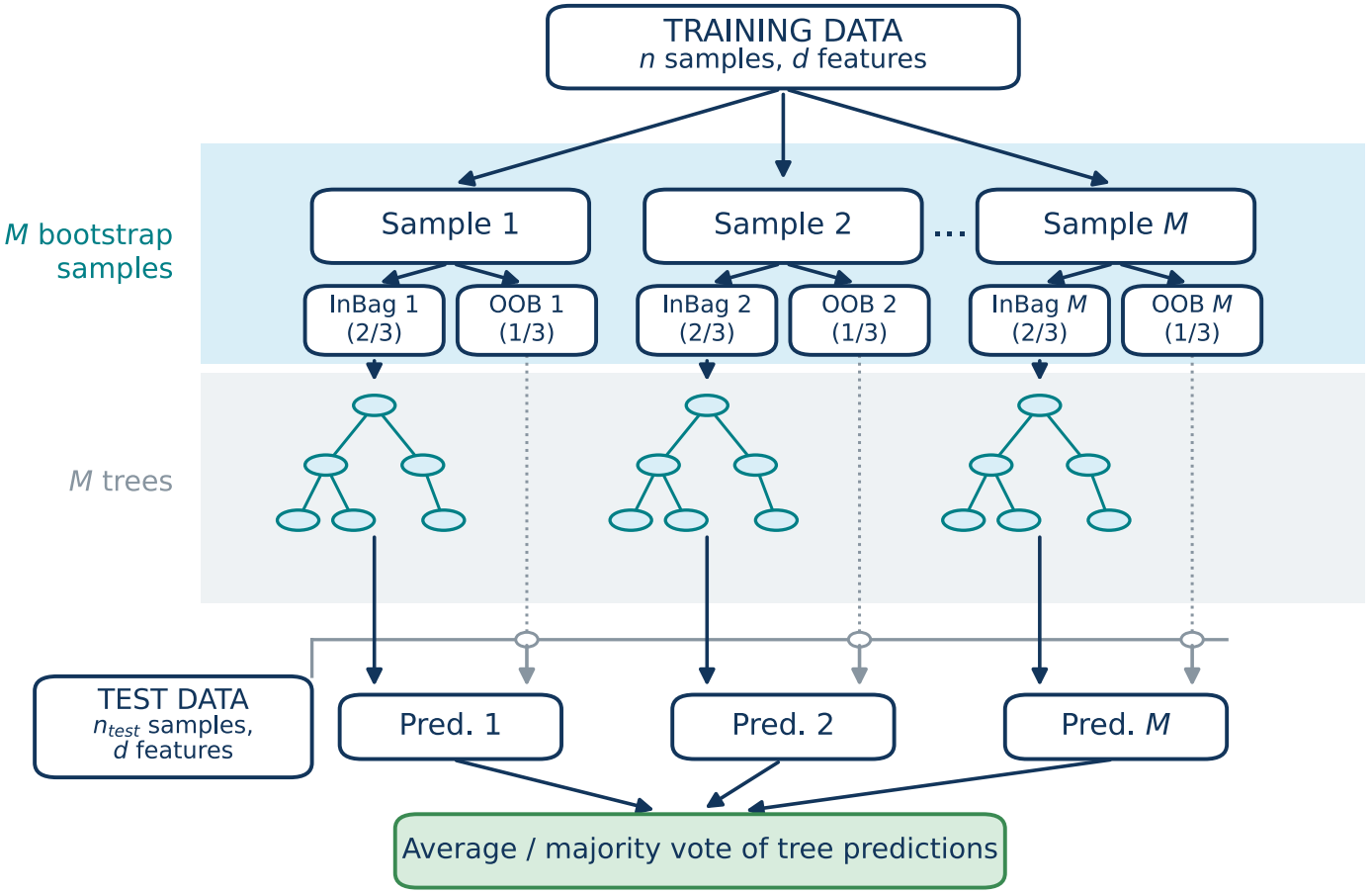


Illustration generated by AI (FLUX.2)

Random forest

- Bootstrap sample (as in bagging).
- **At each split, consider only a random subset of features.**
- **InBag** sample — for training.
- **Out Of Bag (OOB)** samples — the rest, used for validation.

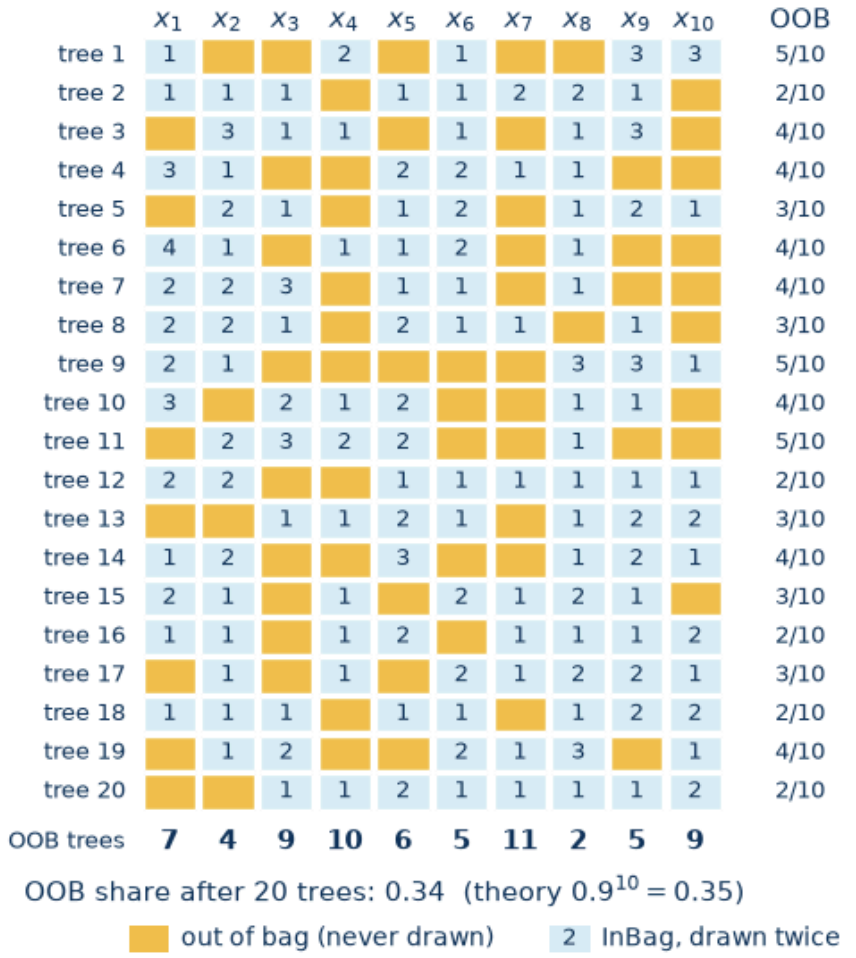
> Two independent sources of randomness: which samples the tree sees, and which features each split may use. Both make the trees less correlated, which is what makes the average worth taking.



Out-of-bag samples

- Testing data for validation — each tree has its own OOB sample.
- Not all trees are used for validation of each sample — only the subset of trees with that sample in their OOB set is used.
- Useful when all the available data are needed for training (small datasets).
- Each sample is scored by only ≈ 1/3 of the trees, so the OOB score tends to be slightly pessimistic — **do not mix OOB and validation scores when comparing models**.

> A bootstrap sample of size n leaves out about $(1 - 1/n)^n \to e^{-1} \approx 37\,\%$ of the data, which is where the "2/3 InBag, 1/3 OOB" rule of thumb comes from. By hand: $n = 10$ gives $0.9^{10} = 0.35$, $n = 100$ gives $0.99^{100} = 0.366$.



code example — OOB score vs. cross-validation

Random forest — conclusion

Pros

- most pros of CART (no scaling, mixed feature types),
- suitable even for small datasets (OOB samples),
- higher performance than a single decision tree,
- performance is comparable with the best standard supervised learning algorithms,
- test error estimate without the need for cross-validation (OOB samples),
- reliable feature importance estimate,
- can be easily parallelized,
- stable and prevents overfitting.

Cons

- less interpretable and intuitive than a single decision tree,
- large number of deep trees — high computational and memory demand in both the training and the prediction phases,
- can still overfit on noisy and/or regression data.



Illustration generated by AI (FLUX.2)

6 · Adaptive boosting (AdaBoost)

> Train the next weak classifier where the previous ones failed, then let them vote with different weights.



Illustration generated by AI (FLUX.2)

Adaptive boosting — AdaBoost

- Linear combination of weak classifiers — **weighted majority vote**.
- Each classifier is trained on a **reweighted version of the data**.
- For binary decision with classes ± 1:

$$H(\mathbf{x}) = \operatorname{sign}\left(\sum_{m=1}^{M} \alpha_m h_m(\mathbf{x})\right)$$

- $\mathbf{x}$ — input data,
- h — weak classifier,
- m — index of weak classifier,
- M — number of weak classifiers,
- H — final strong (boosted) classifier,
- sign — sign function with output $+1$ or -1,
- α — weight of weak classifier — **must be found for each classifier** $\rightarrow$ AdaBoostM1.

The AdaBoost algorithm

Data $T = \{(\mathbf{x}_1, y_1), \ldots, (\mathbf{x}_n, y_n)\}, \mathbf{x}_i \in \mathbb{R}^d, y_i \in \{-1, +1\}$.

1. Initialize sample weights as $w_i^{(m=1)} = \frac{1}{n}$.
2. Find the best weak classifier h_m from the set of all classifiers $\mathcal{H}$ (the one with the lowest weighted error ϵ_m):

$$h_m = \arg\min_{h \in \mathcal{H}} \sum_{i:\, h(\mathbf{x}_i) \neq y_i} w_i^{(m)}, \qquad \epsilon_m = \sum_{i:\, h_m(\mathbf{x}_i) \neq y_i} w_i^{(m)}$$

3. Compute the weight of the weak classifier as $\alpha_m = \frac{1}{2} \ln\left(\frac{1-\epsilon_m}{\epsilon_m}\right)$.
4. Compute new sample weights based on the current weak classifier and its weight:

$$w_i^{(m+1)} = \frac{w_i^{(m)}\, e^{-\alpha_m y_i h_m(\mathbf{x}_i)}}{Z_m}, \qquad Z_m = \sum_{i=1}^{n} w_i^{(m)}\, e^{-\alpha_m y_i h_m(\mathbf{x}_i)} \quad (\text{normalization constant}$$

5. Repeat steps 2.–4. for all M weak classifiers.

AdaBoost example: initialization

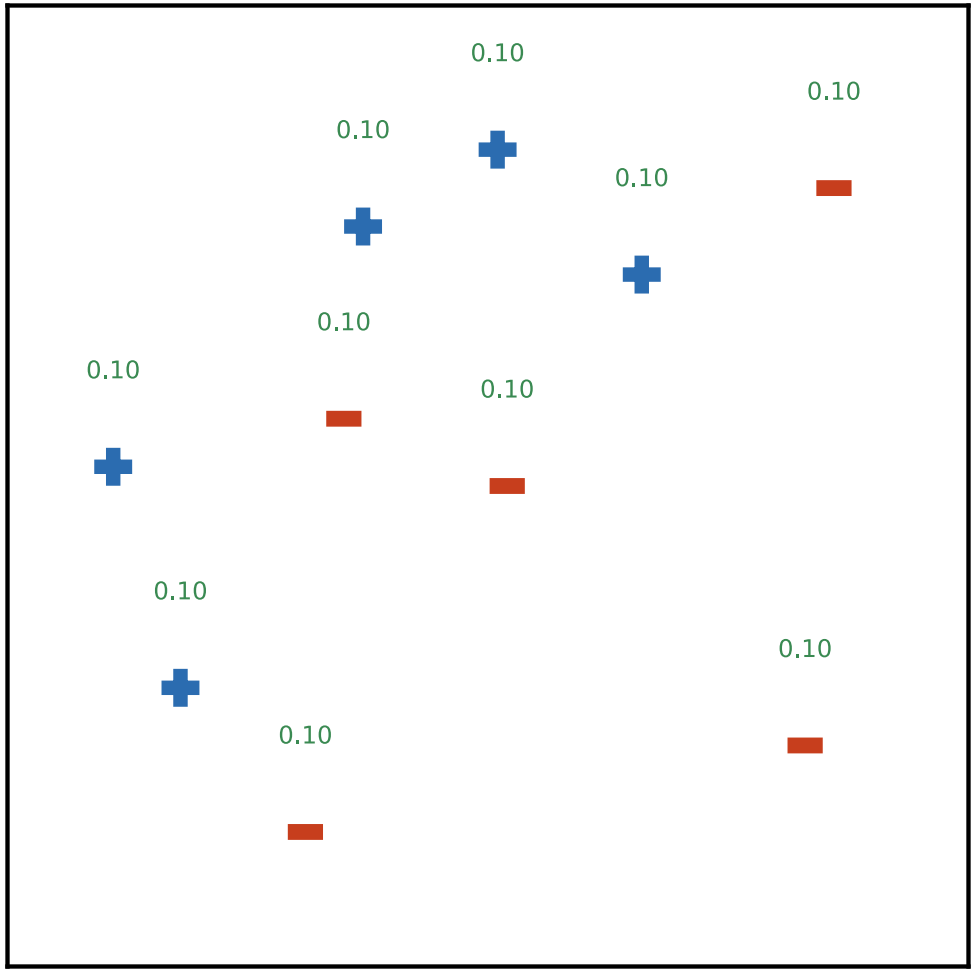
$M = 3$ weak classifiers (decision stumps), $n = 10$ samples, five in class $+1$ (blue) and five in class -1 (red).

All samples start equally important:

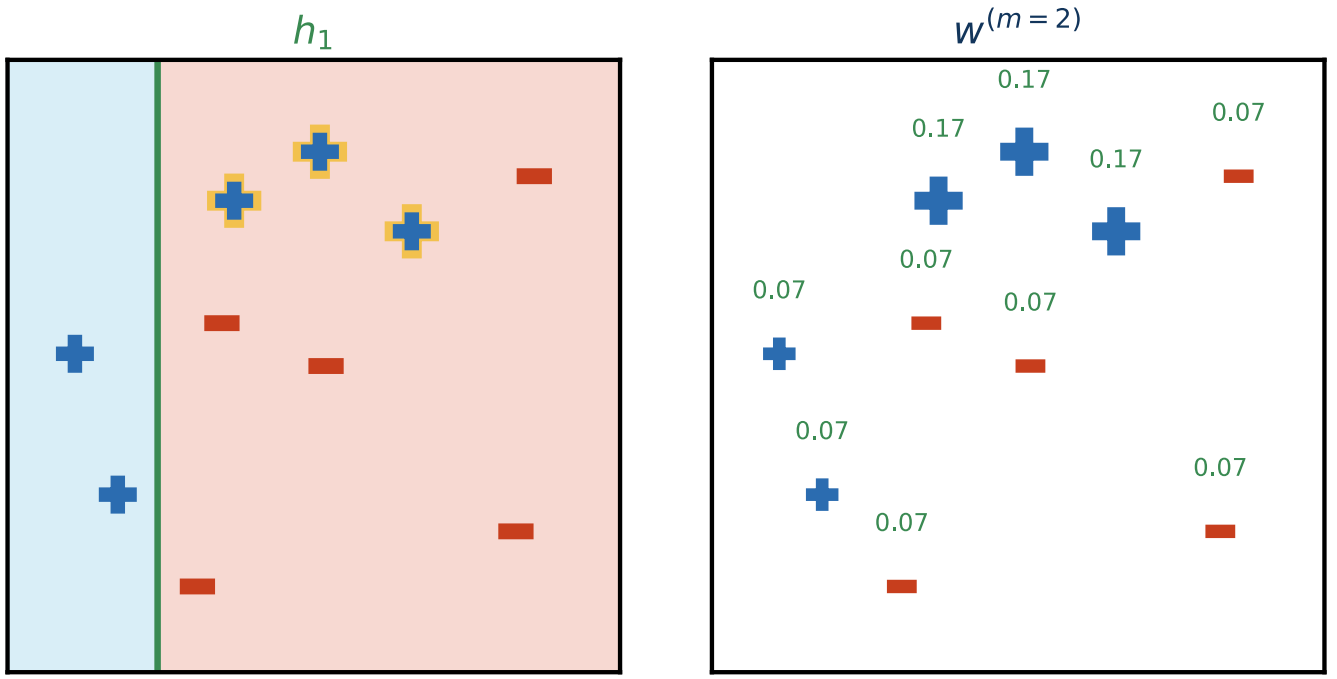
$$w_i^{(m=1)} = \frac{1}{n} = 0.10$$



Illustration generated by AI (FLUX.2)



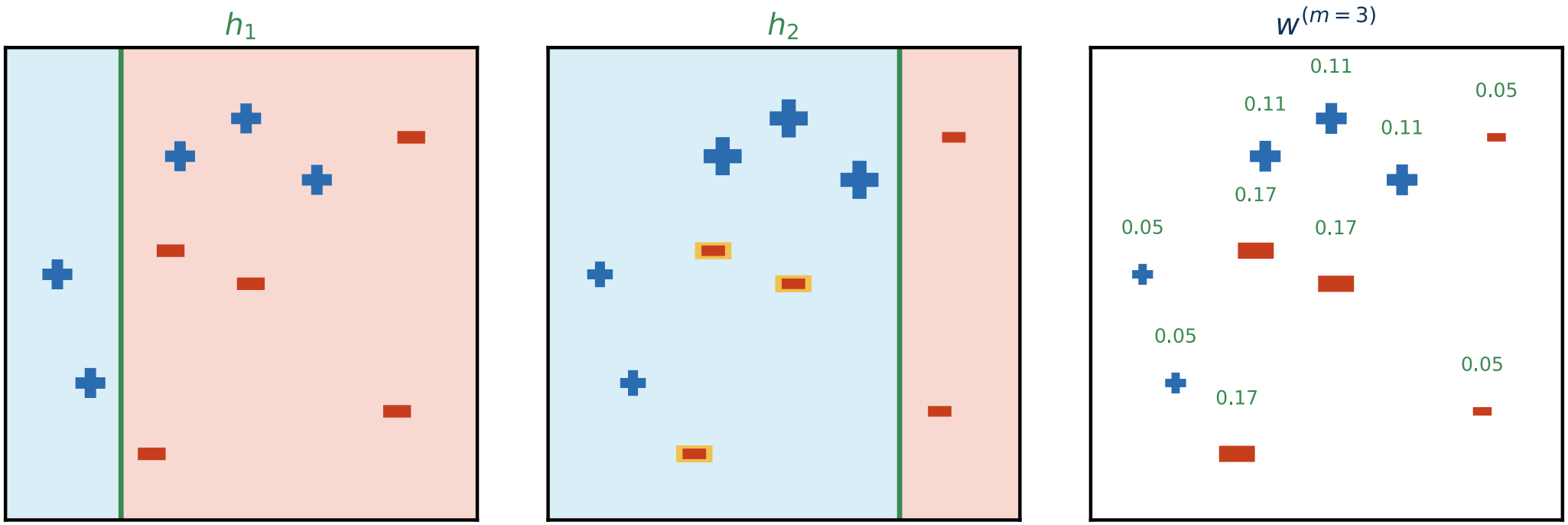
AdaBoost example: the first weak classifier



$\varepsilon_1 = 0.30 \qquad \alpha_1 = 0.42$

Three of ten samples are misclassified (highlighted): $\epsilon_1 = 3 \cdot 0.10 = 0.30, \alpha_1 = \frac{1}{2} \ln \frac{0.70}{0.30} = 0.42$. Misclassified samples get a larger weight, correct ones a smaller one.

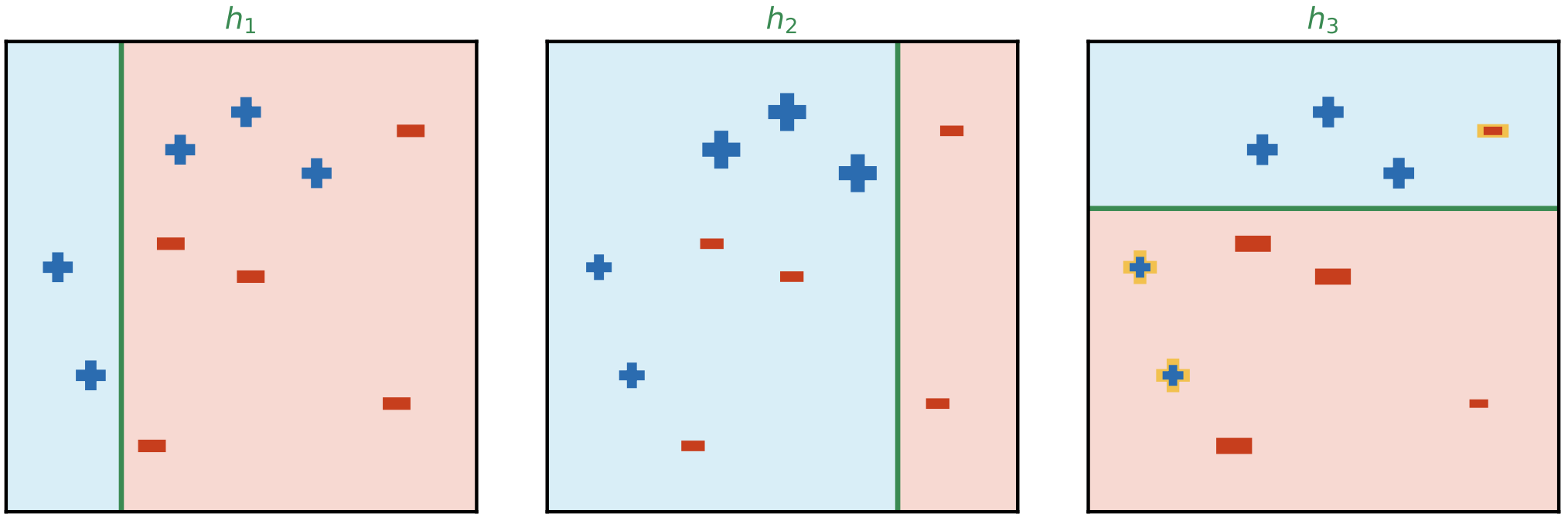
AdaBoost example: the second weak classifier



$\varepsilon_2 = 0.21 \qquad \alpha_2 = 0.65$

The second stump is chosen on the **reweighted** data: it gets the three heavy blue samples right and misclassifies three light red ones, so $\epsilon_2 = 3 \cdot 0.07 = 0.21$ and $\alpha_2 = 0.65 > \alpha_1$.

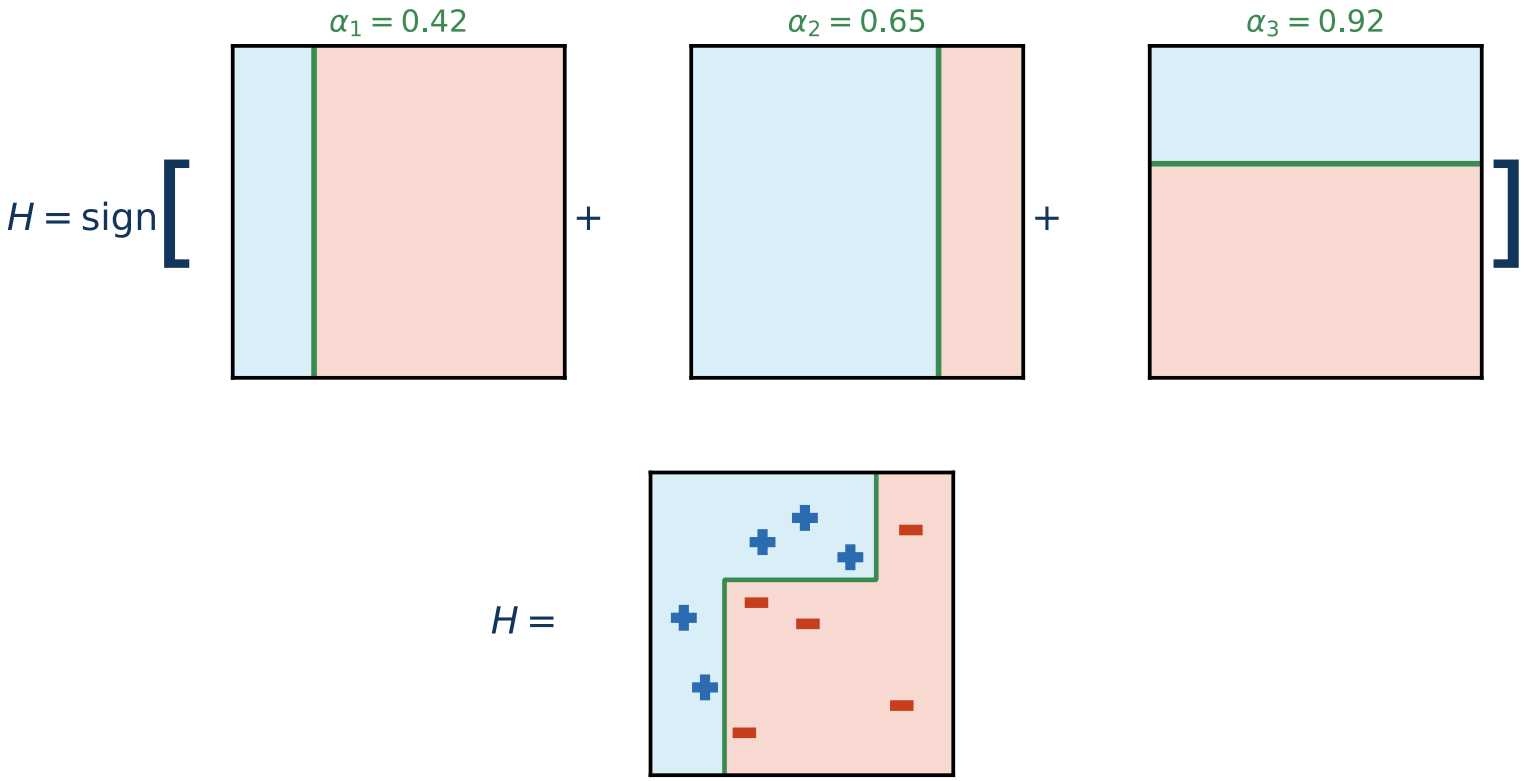
AdaBoost example: the third weak classifier



$\varepsilon_3 = 0.14 \qquad \alpha_3 = 0.92$

The third stump splits **horizontally** and misclassifies three of the four lightest samples (exactly $1/22 = 0.0455$ each, rounded to 0.05 in the figure): $\epsilon_3 = 3 \cdot 0.0455 = 0.14, \alpha_3 = 0.92$.

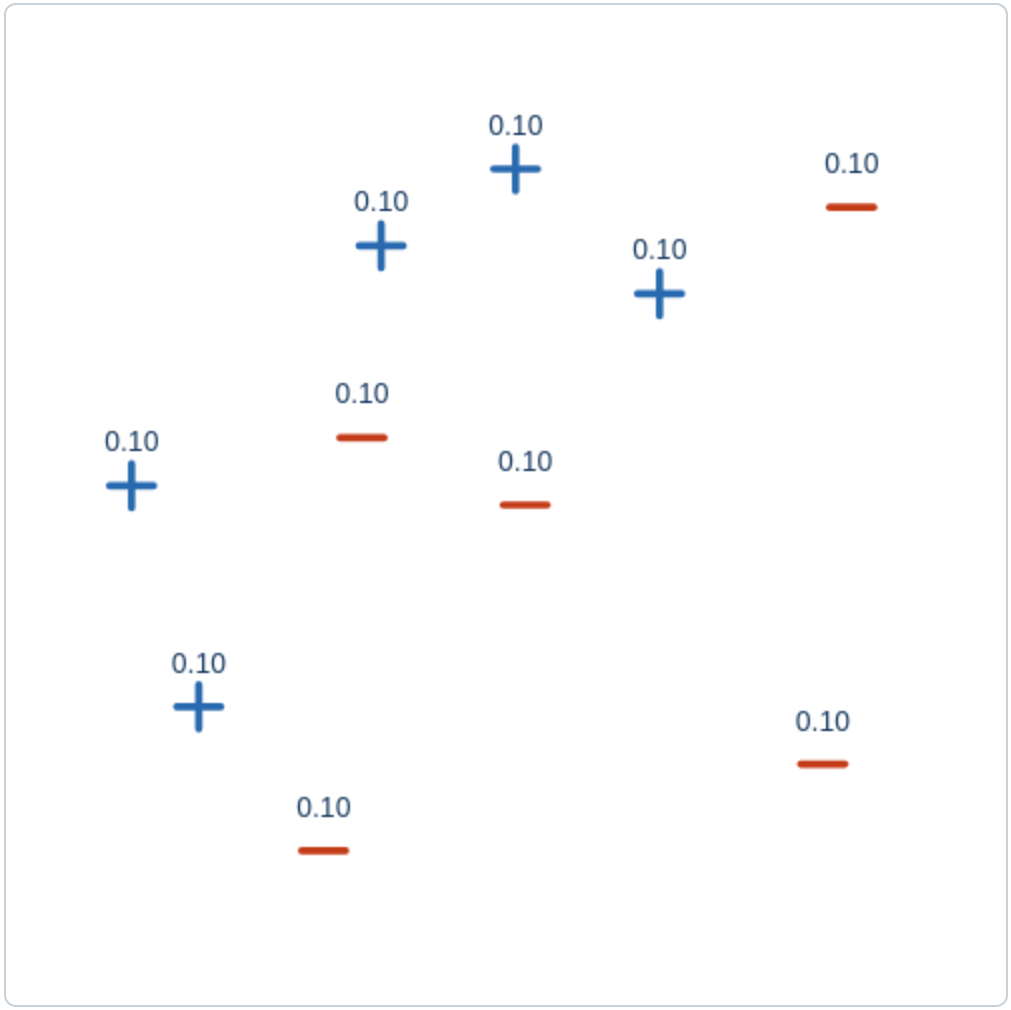
AdaBoost example: the strong classifier



$H(\mathbf{x}) = \text{sign}\big(0.42\, h_1(\mathbf{x}) + 0.65\, h_2(\mathbf{x}) + 0.92\, h_3(\mathbf{x})\big)$ — three axis-aligned stumps combine into a non-trivial decision region that classifies all ten training samples correctly.

code example — AdaBoost from scratch

Playground: AdaBoost round by round



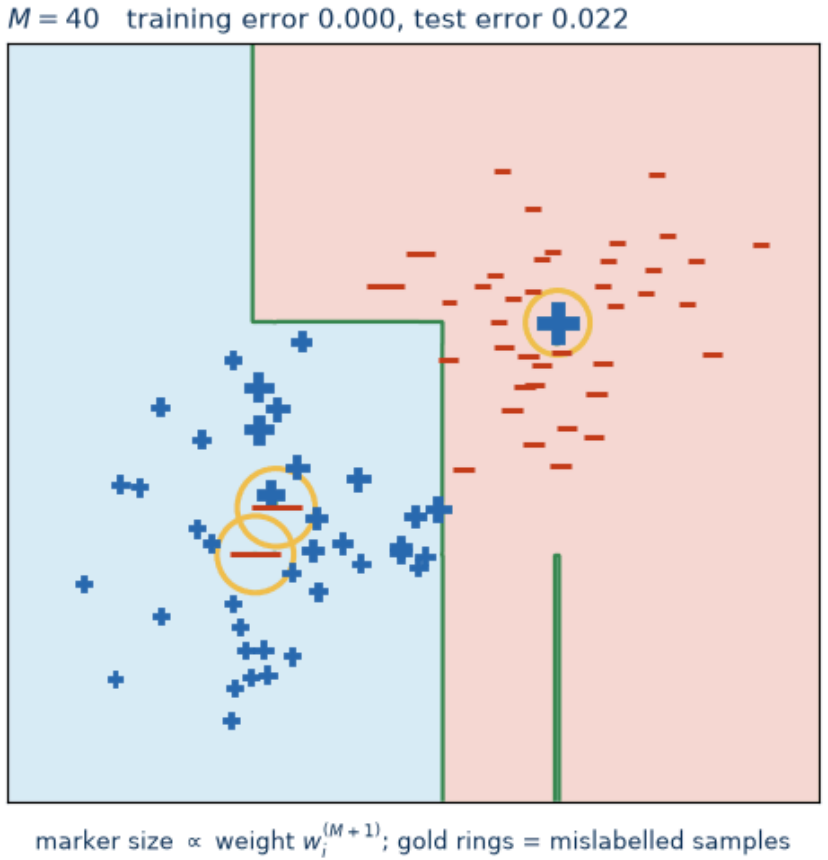
next round | 10 rounds | restart | reset data

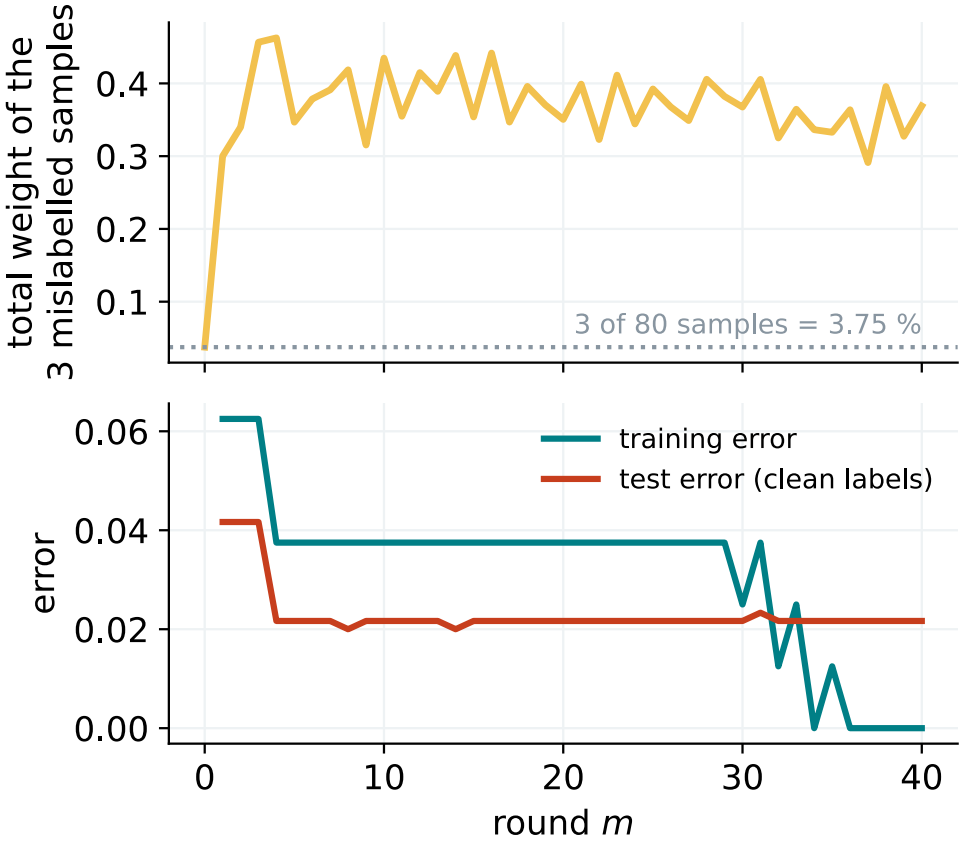
Click a point to flip its label, click elsewhere to add one: ◉ +1 (blue) ○ −1 (red)

Marker area ∝ weight w_i · dashed line = the last stump h_m · background = H(**x**) · gold ring = misclassified by H

0 round(s), n = 10

AdaBoost on noisy labels





Two Gaussian classes, 40 + 40 points, **three labels flipped** (gold rings), decision stumps, $M = 40$. The three mislabelled samples start with $3 \cdot 1/80 = 3.75\,\%$ of the weight and hold **37 %** of it after 40 rounds; the training error reaches 0 — the strong classifier has carved slivers narrower than 0.01 around them (hidden under the markers). Test error on clean labels: $\approx 2\,\%$.

AdaBoost — conclusion

Pros

- fast and simple,
- can combine any ML algorithm,
- can combine expert decisions,
- few hyperparameters: the number of rounds and the type of the weak classifier,
- can be extended for multiclass problems (AdaBoostM2).

Cons

- tends to overfit,
- sensitive to label noise (previous slide).



Illustration generated by AI (FLUX.2)

7 · Other tree algorithms and pruning

> The same idea with a different splitting criterion — and what to do after the tree has grown.



Illustration generated by AI (FLUX.2)

Other tree algorithms and pruning

ID3 (Iterative Dichotomiser 3), Quinlan 1986 — information gain / entropy; works with categorical variables only.

1. Compute total information,
2. compute expected information for each feature,
3. compute information gain for each feature,
4. split with the feature with maximal information gain,
5. recursively repeat 2.–4.

C4.5 / C5.0

- Modification of ID3 — can also use numerical variables.

Pruning

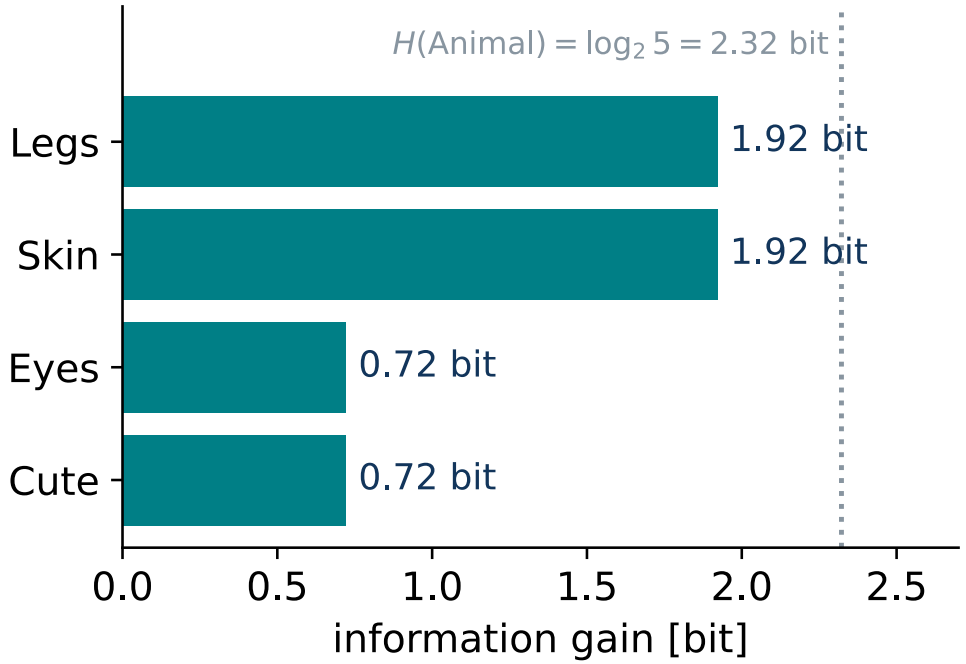
- Grow the full tree first, then cut back the branches that do not improve the validation error — the alternative to stop-splitting rules.

And more: CHAID (Chi-squared Automatic Interaction Detection), MARS (Multivariate Adaptive Regression Splines), etc.

ID3 by hand: which question first?

Target = the animal (5 classes, one sample each), features from the table at the beginning.

1. Total information: $H(\text{Animal}) = \log_2 5 = 2.32$ bit.
2. Expected information after splitting on a feature, e.g. **Legs** $\{4 : \text{Kitten, Zebra}\}$, $\{0\}$, $\{8\}$, $\{2\}$: $\frac{2}{5} \cdot 1 + \frac{1}{5} \cdot 0 + \frac{1}{5} \cdot 0 + \frac{1}{5} \cdot 0 = 0.40$ bit; **Eyes** $\{2 : \text{four animals}\}$, $\{8\}$: $\frac{4}{5} \cdot 2 = 1.60$ bit.
3. Information gain = total − expected: Legs 1.92, Skin 1.92, Eyes 0.72, Cute 0.72 bit.
4. Split on the feature with the largest gain — Legs or Skin (a tie); the tree at the beginning of the lecture asked about legs first.
5. Repeat inside every branch that still mixes classes (here only $\{\text{Kitten, Zebra}\}$).



Animal	Skin	Legs	Eyes	Cute
Kitten	Fur	4	2	True
Snake	Scales	0	2	False
Spider	Chitin	8	8	False
Zebra	Fur	4	2	False
Duck	Feather	2	2	False

8 · Recap

> Three sets of questions from the error decomposition to boosting.



Illustration generated by AI (FLUX.2)

Recap quiz I — bias, variance and noise

- Write the **bias–variance–noise decomposition** of the expected test error. Which of the three terms can we influence and how?
- What exactly is $\bar{h}(\mathbf{x})$ and what is $\bar{y}(\mathbf{x})$? Why is the cross term in the derivation zero?
- You see a large gap between the training and the test error. Which **regime** is it, and what are your three options?
- Your training error is above the acceptable error ϵ. Would more data help? Justify.
- On the error-versus-complexity plot, show the **under-fitting** and **over-fitting** areas, and say what happens to bias and to variance in each.

Recap quiz II — decision trees and CART

- Define **node**, **leaf**, **branch**, **size** and **depth** of a tree. What is a **stump**?
- Write the **splitting criterion** of CART. What exactly is minimized, and over which variables?
- Why is tree growing called **greedy**? What would the alternative cost?
- List the **stop-splitting rules**. What happens if we use none of them?
- Compare **misclassification rate**, **entropy** and **Gini impurity**. Compute all three for a leaf with 4 : 1 samples. Why are the last two preferred in practice?
- How does a **regression** tree differ from a classification tree — criterion, leaf value, shape of the fitted function?
- Name two advantages of trees over linear models in **data preprocessing**.

Recap quiz III — bagging, random forest and AdaBoost

- What is a **bootstrap sample**? Why should averaging M trees reduce the variance by a factor of M — and why did we measure only about 2–3× for $M = 50$? What is the role of the correlation ρ?
- What does **random forest** add on top of bagging, and why does it help?
- Explain **OOB samples**. Why is roughly one third of the data out of bag, and why should OOB and validation scores not be mixed when comparing models?
- Write the **AdaBoost** prediction rule and the formula for α_m. What does α_m do when $\epsilon_m = 0.5$?
- How are the sample weights updated, and what is the role of Z_m?
- **Bagging versus boosting**: which term of the error decomposition does each attack? Which one can be parallelized?
- Why is AdaBoost sensitive to mislabelled samples?